

Unidad 1: Almacenamiento de la información.

Índice de contenidos

1	Introducción.....	2
2	Los ficheros de información.....	2
2.1	¿Qué es un fichero de datos?	2
2.2	Tipos de ficheros.....	3
2.3	Los soportes de información.	4
2.4	Parámetros de utilización.....	4
2.5	Organización de ficheros.	4
2.5.1	Organización secuencial.	5
2.5.2	Organización secuencial encadenada.....	5
2.5.3	Organización secuencial indexada.....	5
2.5.4	Organización directa o aleatoria.....	6
2.6	Inconvenientes de los sistemas de gestión de ficheros.....	8
3	Bases de datos.	9
3.1	Historia y necesidad del uso de las <i>BD</i>	9
3.2	Conceptos y ventajas del uso de <i>BD</i>	10
3.3	¿Quién utiliza las <i>BD</i> y para qué?	10
3.4	Ubicación de la información.....	11
3.4.1	Discos.....	11
3.4.2	Cintas magnéticas.....	11
3.4.3	RAID.	12
3.4.4	Almacenamiento en red.	12
3.4.5	Almacenamiento en la nube.....	13
4	Evolución de los modelos de <i>BD</i>	13
4.1	Modelo jerárquico.	13
4.2	Modelo en red.	14
4.3	Modelo relacional.....	14
4.4	Modelo orientado a objetos.....	14
4.5	Modelo orientado a documentos.....	15
4.6	Otros modelos.	15
4.6.1	Modelo objeto-relacional.....	15
4.6.2	Modelo de <i>BD</i> deductivas o lógicas.....	15
4.6.3	<i>BD</i> multidimensionales.....	16
4.6.4	<i>BD</i> transaccionales.....	16
4.7	Tabla comparativa de los principales modelos.....	16
5	Sistemas Gestores de Bases de Datos (<i>SGBD</i>).	16
5.1	Servicios y funciones proporcionados por un <i>SGBD</i>	17
5.2	Componentes de un <i>SGBD</i>	18
5.3	Arquitectura <i>ANSI/SPARC</i>	18
5.4	Tipos de <i>SGBD</i>	19
6	<i>SGBD</i> comerciales y libres.....	20
7	<i>BD</i> centralizadas.....	20
8	<i>BD</i> distribuidas.....	21
8.1	Fragmentación.....	22
9	Legislación sobre protección de datos.....	26
9.1	Reglamento General de Protección de Datos (<i>RGPD</i>).	27
9.2	Ley Orgánica de Protección de Datos y Garantía de los Derechos Digitales (<i>LOPDGDD</i>).....	28
10	<i>Big Data</i>	30
10.1	Introducción.....	30
10.2	Clasificación de los datos.	30
10.3	Análisis de datos.	31
10.4	Inteligencia de negocios.....	31

1 Introducción.

¿Te has preguntado alguna vez dónde y de qué manera se almacenan y gestionan los datos que se utilizan diariamente? En **casi todas las acciones cotidianas y en la mayoría de los ámbitos de actividad, el uso de bases de datos es esencial**. Éstas, y los datos almacenados en ellas son imprescindibles para realizar muchas actividades.

¿Crees que no es para tanto? Piensa en las siguientes situaciones:

- ✓ Seleccionar un canal favorito en la televisión.
- ✓ Usar la agenda del móvil para hacer una llamada.
- ✓ Operar en un cajero automático.
- ✓ Solicitar un certificado en un organismo público.
- ✓ Asistir a la consulta del médico.
- ✓ Inscribirse en un curso o plataforma en línea.
- ✓ Reservar entradas para un evento deportivo o espectáculo.
- ✓ Consumir ocio digital.
- ✓ Consultar información en Internet (bibliotecas, enciclopedias, museos, etc.).
- ✓ Registrarse en una página de juegos en línea, redes sociales o foros.
- ✓ Incluso, un coche puede incorporar una base de datos.



Es evidente que **casi todo lo que nos rodea, en alguna medida, está relacionado con los datos, su almacenamiento y su gestión**. El gran volumen de datos que actualmente se manejan y sus innumerables posibilidades requieren de técnicos bien formados y capaces de trabajar con ellos.

Este módulo profesional se centra en el **estudio de las Bases de Datos (BD)**.

2 Los ficheros de información.

2.1 ¿Qué es un fichero de datos?

En la década de los setenta, los procesos básicos en las empresas se centraban en la contabilidad y la facturación. Las necesidades de almacenamiento y gestión de información se satisfacían con un número relativamente reducido de archivos en papel, los típicos ficheros clásicos.

En una primera informatización, los datos pasaron del formato papel a ser accesibles más rápidamente a través de ordenadores. La **informática adaptó sus herramientas para que los elementos que el usuario manejaba en el ordenador se parecieran a los manuales**, utilizando términos como ficheros, formularios, carpetas y directorios.

La información se trasladó del papel al formato digital, necesitando almacenarla para su posterior recuperación, consulta y procesamiento. El **elemento que permitió el almacenamiento permanente de datos en dispositivos de memoria masiva** (secundaria o de almacenamiento no volátil) fue el **fichero o archivo**.

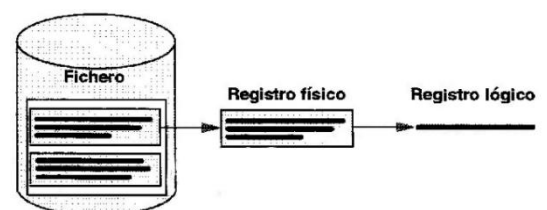
El **sistema de archivos** (sistema lógico de almacenamiento) es una estructura y un conjunto de métodos que un sistema operativo utiliza para organizar, almacenar, recuperar y gestionar archivos en una unidad de almacenamiento, como un disco duro, SSD o una unidad de red.

Ejemplos: FAT, NTFS, ext4, ZFS, XFS, HFS+, APFS, ...

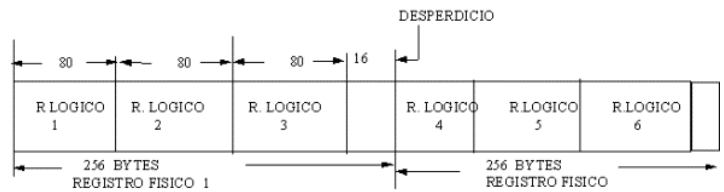
Fichero o archivo de datos: conjunto de información relacionada, tratada como un todo y organizada de forma estructurada para permitir la búsqueda de datos individuales. Son identificados por un nombre y la ubicación de la carpeta o directorio que los contiene.

Los **ficheros de datos** están **formados por registros lógicos** que **contienen datos relativos a un mismo elemento u objeto** (como usuarios o clientes). Estos **registros** están **divididos en campos** que **contienen las informaciones elementales** que forman el registro (nombre del usuario, dirección de correo electrónico, teléfono, etc.).

Dado que los **ficheros suelen ser muy voluminosos**, solo se pueden **cargar parte de ellos en la memoria principal** para su procesamiento. La **cantidad de información que el Sistema Operativo (SO) transfiere entre el soporte en el que se almacena el fichero y la memoria principal del ordenador en una sola operación de lectura/escritura se denomina registro físico o bloque**.

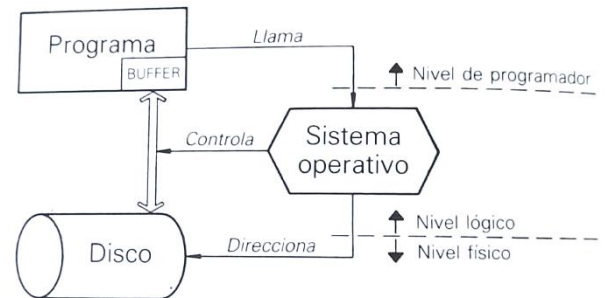


Normalmente, en cada operación de lectura/escritura se transfieren varios registros lógicos del fichero; es decir, un bloque suele contener varios registros lógicos. El **número de registros lógicos que caben en un bloque** se conoce como **factor de blocaje**, y la **operación de agrupar varios registros en un bloque** se llama **bloqueo de registros**.



El SO también convierte la dirección lógica (la posición relativa del registro en el fichero) utilizada en los programas de usuario en la dirección física (la posición real del registro en el soporte de información) con la que se direcciona en el soporte.

Una **clave** o identificador es un **campo (o conjunto de campos) que identifica cada registro del fichero**. En un mismo fichero puede haber una, varias o ninguna clave. Cuando una **clave se utiliza como campo de localización en el fichero**, se denomina **llave (key)**.



Los **ficheros pueden estar formados por registros** de los siguientes tipos:

- ✓ **Longitud fija:** la suma de las longitudes de cada campo (en caracteres) es siempre la misma.
- ✓ **Longitud variable:** las longitudes de los campos no siempre son las mismas. El sistema reserva una **palabra al comienzo de cada registro para anotar su longitud**.
- ✓ **Delimitados:** el sistema **incluye un carácter especial al final de cada registro**.
- ✓ **Indefinido:** en este caso, el SO no gestiona la longitud de los registros del fichero. **El programa del usuario se encargará de localizar el principio y el final de cada registro**.

2.2 Tipos de ficheros.

En una aplicación informática, los ficheros se utilizan para realizar diversas funciones. Conocer la función de un fichero concreto es fundamental para decidir cómo organizarlo. **Según su función, los ficheros pueden clasificarse en:**

- **Ficheros permanentes:** contienen **información relevante para la aplicación**, necesaria para su funcionamiento. **Su vida es larga** y generalmente **no pueden generarse de forma inmediata a partir de otros ficheros**. Estos se subdividen en:
 - ✓ **Ficheros maestros:** contienen el **estado actual de los datos que pueden modificarse** desde la aplicación. Son el **núcleo de la aplicación**, como un archivo con los datos de los usuarios de una plataforma educativa o un archivo de clientes de un banco.
 - ✓ **Ficheros constantes:** **incluyen datos fijos para la aplicación que rara vez se modifican y se utilizan principalmente para consultas**, como un archivo con códigos postales, la ubicación de estantes en una biblioteca o una tabla de números primos.
 - ✓ **Ficheros históricos:** contienen **datos que fueron considerados actuales en un periodo anterior** y se utilizan para reconstruir situaciones pasadas o presentes, como un archivo con los usuarios dados de baja en una plataforma educativa.
- **Ficheros temporales:** **almacenan información útil solo para una parte de la aplicación y se generan a partir de los datos de los ficheros permanentes**. Tienen un **corto periodo de existencia**. Estos se subdividen en:
 - ✓ **Ficheros intermedios:** almacenan **resultados de una aplicación que serán utilizados por otra dentro de una misma tarea**.
 - ✓ **Ficheros de maniobras:** almacenan **datos de una aplicación que no pueden mantenerse en la memoria principal por falta de espacio**.
 - ✓ **Ficheros de resultados:** almacenan **datos que serán transferidos a un dispositivo de salida**.



Autoevaluación

Imaginar una aplicación diseñada para gestionar una biblioteca. En esta aplicación, habrá un fichero que contenga el catálogo de libros disponibles, otro que almacene información sobre las editoriales y otro que registre los libros que han quedado obsoletos. ¿A qué tipo correspondería el fichero que almacena las editoriales?

- ☐ Fichero maestro.
- ☐ Fichero constante.
- ☐ Fichero intermedio.

2.3 Los soportes de información.

Los **ficheros se almacenan en soportes de información** manejados por dispositivos periféricos del ordenador, que permiten leer y grabar datos en el soporte. Los **soportes más utilizados** son las **cintas magnéticas** y los **discos** (magnéticos, ópticos, magneto-ópticos o de estado sólido). Existen en el mercado una gran variedad de modelos de estos tipos de soportes. El **tamaño de los ficheros está limitado por el tamaño de los dispositivos que los albergan y el sistema de archivos implantado** (por ejemplo, FAT32 no permite más de 4 GB).



Inicialmente, los primeros sistemas de almacenamiento físico fueron los tambores de cinta magnética, de dimensiones parecidas a los discos de vinilo. Estos tambores funcionaban de manera similar a los antiguos casetes, pero sus mayores dimensiones les permitía almacenar gran cantidad de datos en formato digital (0 y 1) en orden secuencial.

Posteriormente, los sistemas de almacenamiento de información evolucionaron con los avances en el hardware, particularmente con la aparición del disquete y el disco duro. Estos dispositivos permiten el acceso aleatorio, lo que significa que no es necesario pasar por todos los datos desde el inicio hasta la zona donde se encuentra la información deseada.



Por tanto, **se distinguen dos tipos principales de soportes para el almacenamiento de datos:**

- ✓ **Soportes de acceso secuencial o no direccionables** (como las cintas magnéticas): se utilizan principalmente para copias de seguridad. **Si se necesita acceder a un dato** en una posición específica, **es necesario leer todos los datos que haya hasta llegar a esa posición**.
- ✓ **Soportes de acceso directo o direccionables** (como los discos): son los más utilizados y **permiten acceder directamente a la posición deseada** y leer los datos desde allí, **sin necesidad de recorrer o leer otros bloques**.

2.4 Parámetros de utilización.

Para determinar la organización más adecuada de un fichero, es importante **considerar sus parámetros de referencia**:

- a) **Capacidad o volumen**: es el **espacio** en bytes (o sus múltiplos) **que ocupa el fichero**. Se calcula multiplicando el número previsto de registros por la longitud media de cada registro.
- b) **Actividad**: permite conocer la **cantidad de consultas y modificaciones en el fichero**. Se determina mediante:
 - **Tasa de consulta o modificación**: el **porcentaje de registros consultados o modificados en cada tratamiento** del fichero **respecto al número total de registros**.
 - **Frecuencia de consulta o modificación**: el **número de veces que se accede al fichero para realizar una consulta o modificación en un periodo de tiempo fijo**.
- c) **Volatilidad**: mide la cantidad de **inserciones y borrados** que se efectúan **en un fichero**. Se determina con:
 - **Tasa de renovación**: el **porcentaje de registros renovados en cada tratamiento** del fichero **respecto al número total de registros**.
 - **Frecuencia de renovación**: el **número de veces que se accede al fichero para renovarlo en un periodo de tiempo fijo**.
- d) **Crecimiento**: es la **variación de la capacidad del fichero** y se mide con la **tasa de crecimiento**, que es el **porcentaje de registros en que aumenta el fichero en cada tratamiento** con respecto al total de registros.



Autoevaluación

La volatilidad de un fichero es un parámetro que indica:

- La variación del volumen del fichero.
- La cantidad de veces que se abre o cierra el fichero.
- La cantidad de inserciones y borrados en dicho fichero.

2.5 Organización de ficheros.

Hay **varias formas de organizar los ficheros en un soporte de información**, y las características de uso del fichero dependen de la organización adoptada. Por lo tanto, **la elección de la organización debe basarse en la forma en que**

se va a usar el fichero. A continuación, se detallan las principales formas de organizar los ficheros.

2.5.1 Organización secuencial.

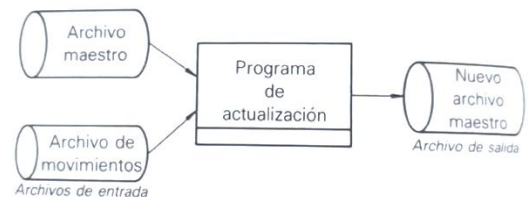
Un fichero con organización secuencial almacena **sus registros de forma contigua**, lo que significa **que solo se puede acceder a ellos leyendo uno tras otro desde el principio hasta el final**. Por lo general, estos ficheros tienen una marca indicativa del fin del fichero, conocida como *EOF* (*End of File*).

Características de los ficheros con organización secuencial:

- ✓ La **lectura siempre se realiza hacia delante**.
- ✓ Tienen una **estructura rígida de campos**, es decir, la posición de los campos de cada registro siempre ha de ser la misma.
- ✓ Son ficheros **monousuario**, es decir, no permiten el acceso simultáneo de varios usuarios.
- ✓ **Aprovechan al máximo el soporte de almacenamiento**, ya que no dejan huecos vacíos.
- ✓ **Se pueden almacenar en cualquier tipo de soporte**, ya sea **secuencial** o **direccionable**.
- ✓ **Todos los lenguajes de programación ofrecen instrucciones para trabajar con este tipo de ficheros**.
- ✓ Las **consultas se realizan en orden secuencial**, leyendo los registros uno tras otro.
- ✓ **No se pueden insertar registros entre los que ya están grabados**.

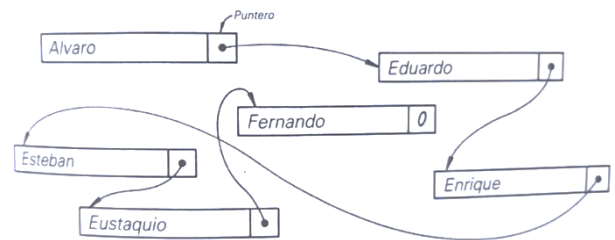


Realizar operaciones como la inserción, modificación o eliminación de registros en un archivo secuencial no es una tarea sencilla. Cuando se necesita actualizar un archivo de este tipo, es necesario crear uno nuevo, proceso que debe llevarse a cabo mediante un programa creado para tal fin. La única operación que puede realizarse fácilmente es la de añadir nuevos registros al final del archivo. Las consultas en este tipo de archivos se efectúan de manera secuencial, recorriendo los registros en el orden en que fueron guardados.



2.5.2 Organización secuencial encadenada.

En un fichero con organización secuencial encadenada, junto a cada registro se almacena un puntero (enlace) que indica la dirección del registro siguiente según el orden lógico del fichero. Este tipo de ficheros **sólo pueden ser gestionados en soportes direccionables**.



La **consulta** en este tipo de ficheros es **secuencial**, cada vez que se lee un registro, se lee también el puntero que indica la posición del siguiente, lo que permite seguir la secuencia lógica del archivo.

Para **insertar un registro**, primero se localiza la posición deseada entre dos registros existentes. Luego, se crea el nuevo registro en un espacio libre y se modifican los punteros para insertar el registro entre los dos deseados.

Para **borrar un registro**, se asigna al puntero del registro anterior la dirección del siguiente al que se desea borrar. El SO puede o no liberar el espacio ocupado por el registro eliminado.

En caso de **modificar un registro** sin aumentar su longitud, se puede **reescribir en el mismo espacio**. Si la modificación implica un aumento en la longitud del registro, se debe insertar el nuevo registro modificado y luego borrar la versión anterior a la modificación.

La principal **ventaja** de esta organización es su **flexibilidad** (para insertar, modificar o eliminar registros), mientras que su principal **desventaja** es su limitación a la consulta secuencial.

2.5.3 Organización secuencial indexada.

Un fichero con organización secuencial indexada está formado por tres áreas o zonas distintas:

- ✓ **Zona de registros** (área primaria o de datos): aquí se graban los registros en un soporte de almacenamiento de acceso directo, en secuencia ascendente según los valores de la clave y en bloques de longitud fija.
- ✓ **Zona de índices** (área de índices): esta área se procesa secuencialmente y contiene una **tabla que relaciona las claves con las direcciones de los registros en la zona de registros**. Cada entrada contiene el valor más alto de la

área primaria			
nº registro	clave	nombre	apellidos
1	20	Pierre	Llontop
2	34	Adela	Poyet
3	75	Martin	Romero
4	90	Abril	Sierra
5	102	Luis	Serrano
6	108	Maite	Cascón
7	123	Juan	Torres
8	260	Mikel	Artetxe
9	315	Rosa	Bueno

área desbordamiento			
clave	nombre	apellidos	
190	Tomás	Heintz	
314	Luis	Bayón	

área de índices			
clave último elemento	primer índice		
75	1		
108	4		
315	7		

clave de cada tramo de registros y la dirección del primer registro del tramo.

- ✓ **Zona de desbordamiento** (área de desbordamiento o excedentes): aquí **se graban los registros que no caben en la zona de registros**. Los registros en esta área están desordenados y cada nuevo registro se escribe al final.

La zona de registros está dividida en tramos lógicos, cada uno formado por una serie de registros consecutivos. Por cada tramo en la zona de registros, hay un registro correspondiente en la zona de índices. Este registro indica el valor de la clave del último elemento del tramo y la posición por la que comenzar para recorrer el tramo.

Esta organización es muy **utilizada** tanto para **procesos con pocos registros** como para **aquellos que implican el manejo completo del fichero**.

Las principales **características** de la organización secuencial indexada son:

- ✓ **Permite el acceso secuencial**, los registros se leen en orden según el campo clave.
- ✓ **Permite el acceso directo a los registros**, primero busca la clave en el área de índices y luego lee los datos en la dirección indicada por la tabla.
- ✓ **Permite la actualización de registros en el mismo fichero**, no es necesario crear uno nuevo para la actualización.
- ✓ **Requiere más espacio en el disco que los ficheros secuenciales**, debido al uso del área de índices.
- ✓ **Solo se puede utilizar con soportes direccionables**.

2.5.4 Organización directa o aleatoria.

En este tipo de ficheros, **se puede acceder a un registro indicando la posición relativa del mismo dentro del archivo** o, más comúnmente, **mediante una clave** que forma parte del registro como un campo más. Estos archivos **deben almacenarse en dispositivos de almacenamiento de acceso directo** (discos).

Campo clave: campo que permite identificar y localizar un registro de forma rápida y organizada.

Cada registro se guarda en una posición física en el disco, dependiendo del espacio disponible. Por eso, **la distribución de los registros es aleatoria dentro del almacenamiento**. Para acceder a la posición física de un registro, se usa una dirección o índice, sin necesidad de recorrer todo el fichero para encontrar un registro específico.

Algunas **características** fundamentales de los ficheros con organización directa o aleatoria son:

- ✓ **Posicionamiento inmediato**.
- ✓ **Registros de longitud fija**.
- ✓ **Permiten su uso por múltiples usuarios**.
- ✓ Los **registros se borran colocando un cero en la posición que ocupan**.
- ✓ Permiten la utilización de **algoritmos de compactación de huecos**.
- ✓ **Solo son posibles en soportes direccionables**.
- ✓ **Se crean con un tamaño definido y un máximo de registros preestablecido**.
- ✓ Se usan cuando el acceso a los datos se realiza siempre utilizando la misma clave y se requiere **alta velocidad de acceso**.
- ✓ **Permiten la actualización de registros en el mismo fichero**, sin necesidad de copiarlo.
- ✓ Permiten la **actualización de registros en tiempo real**.

2.5.4.1 Direccionamiento directo.

Es **factible cuando la clave es numérica** y su rango de valores no supera el rango de direcciones del archivo. En este caso, **la propia clave se usa como dirección**.

A veces, **pueden quedar lagunas de direcciones sin usar**, en lugares conocidos de antemano.

Un archivo aleatorio con direccionamiento directo **siempre está ordenado por la clave**.

Ejemplo: suponer que se tiene un fichero de datos para almacenar información de empleados. Cada empleado tiene un número de identificación único que se usará como clave numérica para el direccionamiento directo.

- Estructura del fichero: el fichero tiene una estructura predefinida en la que cada registro ocupa una posición fija. Por ejemplo, si cada registro ocupa 100 bytes, el primer registro estará en las posiciones 0-99, el segundo registro en las posiciones 100-199, y así sucesivamente.

- Asignación de claves: cada empleado recibe un número de identificación único que servirá como clave numérica para acceder a su registro en el fichero.
- Direccionamiento directo: para acceder a un registro de empleado, se usa su número de identificación como clave para calcular su ubicación física en el fichero. Por ejemplo, el empleado con identificación 1002 tendrá su registro en la posición 100100-100199.

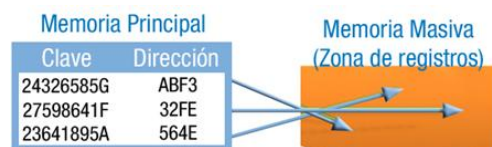
$$\text{posición} = (\text{número_identificación} - 1) * 100$$

El direccionamiento directo con clave numérica permite acceder directamente a un registro en el fichero utilizando la clave numérica como índice para calcular su ubicación física. Esto evita la necesidad de buscar secuencialmente a través de los registros, lo que proporciona un acceso rápido y eficiente a los datos.

Es crucial que el fichero esté organizado de manera que los registros tengan una ubicación fija y predecible basada en la clave numérica para que el direccionamiento directo funcione correctamente.

2.5.4.2 Direccionamiento asociado.

Se puede utilizar para cualquier tipo de clave. Con este método, se construye una tabla que incluye todas las claves y las direcciones de los registros correspondientes.



Operaciones relacionadas:

- ✓ **Creación** del fichero: al crear el fichero de datos, se asigna un identificador único a cada registro. Además, **se mantiene una tabla de índices** (se requiere una estructura adicional) **que almacena la relación entre la clave y la posición del registro en el fichero.**
- ✓ **Inserción** de registros: al insertar un nuevo registro, **se le asigna una clave única y se guarda en la siguiente posición disponible.** Luego, **se actualiza la tabla de índices** para asociar la clave con la posición del registro en el fichero. Al **añadir nuevos registros** las **claves se colocan al final de la tabla**, normalmente desordenada.
- ✓ **Búsqueda** de registros: **se busca la clave en la tabla de índices para obtener la posición del registro en el fichero, y luego se accede directamente a esa posición** para recuperar el registro correspondiente. Cargar en memoria principal u ordenar la tabla de índices para mejorar la velocidad de acceso.
- ✓ **Actualización** de registros: para actualizar un registro, **se utiliza la clave para buscar su posición en la tabla de índices.** Una vez localizada, **se actualiza el registro directamente en esa posición en el fichero.**
- ✓ **Borrado** de registros: **marcar los registros como borrados y actualizar la tabla de índices** según sea necesario. El espacio del registro marcado como borrado puede reutilizarse en el futuro, y puede ser necesaria una limpieza periódica para eliminar físicamente los registros marcados como borrados y recuperar espacio en el archivo.

El direccionamiento asociado permite un **acceso directo a los registros utilizando la clave correspondiente**, evitando la necesidad de recorrer todo el fichero para buscar el registro deseado. Esto resulta en un acceso más rápido y eficiente. Además, la tabla de índices facilita el mapeo de las claves a las posiciones físicas en el fichero, simplificando la recuperación y actualización de los registros.

2.5.4.3 Direccionamiento calculado (hashing).

La técnica del acceso calculado o *hashing* permite accesos más rápidos al **utilizar una función matemática conocida (función de hashing) que genera la dirección de cada registro del archivo a partir de la clave.** Si la clave es alfanumérica, primero debe transformarse en un número.

Surgen dos **situaciones indeseadas**:

- ✓ **Direcciones que no corresponden a ninguna clave** (zonas de disco sin usar).
- ✓ **Direcciones que corresponden a más de una clave.**

El mayor **problema** es la **posibilidad de obtener la misma dirección a partir de diferentes claves al aplicar la función matemática**, conocido como **colisión**. Las **claves que generan la misma dirección se llaman sinónimos**.

Hay **dos formas de resolver el problema de los sinónimos**, aunque complican la estructura del fichero:

1. Si una clave se asocia a una dirección ya ocupada por otro registro, **se busca en el archivo por algún procedimiento una posición libre donde escribir el nuevo registro.**
2. **Se reserva una zona de desbordamiento para los registros que no puedan escribirse en su posición asignada por la transformación.**

Una **buena función de hash minimiza las colisiones**. Esta función implica realizar cálculos matemáticos con el valor de la clave hasta obtener un número entre 1 y n , siendo n el número de direcciones que tiene el fichero.

Algunos **métodos** para generar las direcciones son:

- ✓ **Módulo:** la dirección es el **resto de la división entera entre la clave y el número de registros**.
- ✓ **Extracción:** la dirección es una **parte de las cifras extraídas de la clave**.
- ✓ **Elevación al cuadrado:** se **eleva al cuadrado la clave y se toman los dígitos centrales**.
- ✓ **Plegamiento:** la **clave se descompone en segmentos de cifras contiguas del mismo tamaño y se suman**.

Ejemplo: suponer que se tiene un fichero de datos que almacena información de productos. Cada registro del fichero contiene un campo clave, como el código de producto, y otros campos con información relevante sobre el producto.

- Función de *hash*: se define una función de *hash* que toma el código de producto y genera un número *hash* único correspondiente a ese producto. Por ejemplo:

$$\text{valor_hash} = \text{código_producto} \% \text{tamaño_fichero}$$

- Direccionamiento: para insertar un nuevo producto en el fichero de datos, se calcula su número *hash* utilizando la función de *hash* y se utiliza ese valor como dirección para ubicar el registro en el fichero. Por ejemplo:

$$\text{valor_hash} = \text{hash}("P12345") \% \text{tamaño_fichero}$$

Luego, se inserta el registro del producto en la posición calculada en el fichero de datos.

- Búsqueda: para buscar un producto en el fichero de datos, se calcula su número *hash* utilizando la función de *hash* y se accede a esa dirección para obtener el registro correspondiente. Por ejemplo:

$$\text{valor_hash} = \text{hash}("P67890") \% \text{tamaño_fichero}$$

Luego, se accede al registro en la posición calculada en el fichero y se obtiene la información del producto.



Autoevaluación

En los ficheros de acceso directo los registros siempre se encuentran en posiciones contiguas dentro del soporte de almacenamiento. ¿Verdadero o falso? Razona la respuesta.

2.6 Inconvenientes de los sistemas de gestión de ficheros.

Antes de la llegada de los SGBD (Sistemas Gestores de Bases de Datos), los datos se almacenaban en ficheros y los programas los manipulaban para obtener la información. Si la estructura de los datos cambia en los ficheros, todos los programas que los manejan se deben modificar. En estos sistemas de gestión de ficheros, **la definición de los datos se encuentra codificada dentro de los programas en lugar de almacenarse de forma independiente**, y además **el control del acceso y la manipulación de los datos vienen impuestos por los programas**.

Esto supone un gran inconveniente a la hora de tratar grandes volúmenes de información. **Surge así la idea de separar los datos contenidos en los ficheros de los programas que los manipulan**, es decir, que se pueda modificar la estructura de los datos de los ficheros sin que por ello se tengan que modificar los programas.

Inconvenientes de los sistemas de gestión de ficheros:

- ✓ **Redundancia e inconsistencia de los datos:** los ficheros son creados por diferentes programas y van cambiando con el tiempo, por lo que pueden tener distintos formatos y los datos pueden estar duplicados.
- ✓ **Dependencia física-lógica de los datos:** la estructura física de los datos (definición de ficheros y registros) está codificada en los programas.
- ✓ **Dificultad para acceder a los datos:** cada vez que se necesita una consulta no prevista es necesario codificar el programa correspondiente.
- ✓ **Separación y aislamiento de los datos:** al estar dispersos en varios ficheros con diferentes formatos, resulta complicado escribir nuevos programas que aseguren la correcta manipulación de los datos.
- ✓ **Dificultad para el acceso concurrente:** es complejo permitir que varios usuarios actualicen los datos al mismo tiempo.
- ✓ **Dependencia de la estructura del fichero con el lenguaje de programación:** la estructura está definida dentro de los programas, la incompatibilidad entre ficheros generados por distintos lenguajes de programación hace que los datos sean difíciles de procesar.
- ✓ **Problemas de seguridad de los datos:** la implementación de restricciones de seguridad es difícil, ya que las

aplicaciones se van añadiendo al sistema según se van necesitando.

- ✓ **Problemas de integridad de datos:** los valores almacenados en los ficheros deben cumplir con restricciones de consistencia, como no crear un pedido si el cliente no existe.

Todos estos inconvenientes impulsaron el desarrollo de las **BD** y los **SGBD**.



TAREA

1. ¿Es posible desarrollar una aplicación para el control de las ventas de ordenadores de una tienda de informática sin hacer uso de un **SGBD**? ¿de qué recurso se dispone para almacenar la información? ¿qué problemas va a plantear este desarrollo? ¿de qué tipos pueden ser los elementos utilizados para guardar la información? Poner un ejemplo de la información que se guardaría en cada uno de los tipos.
2. ¿Qué es un “fichero de datos”? Crear una tabla comparativa con las ventajas e inconvenientes de las diferentes formas de organizar los ficheros de datos.
3. Investigar en *Internet* sobre los sistemas lógicos de almacenamiento (sistemas de archivos o ficheros) ¿Qué son? ¿qué características presentan? ¿cuáles son los más utilizados en la actualidad?

3 Bases de datos.

Las **BD** y los **SGBD** surgen para resolver los problemas asociados con la dependencia directa de las aplicaciones de los archivos (duplicación de información, incoherencia de datos, fallos de seguridad, etc.).

Una **BD** permite reunir toda la información relacionada en un mismo sistema de almacenamiento, lo que permite que cualquier aplicación la utilice de manera independiente. Además, mejora el tratamiento de la información y facilita el desarrollo de aplicaciones.

Las **BD** son útiles para gestionar grandes volúmenes de datos y facilitar la consulta de información. También permiten definir permisos para regular el acceso a los datos.

Es importante distinguir entre la **BD** en sí y el sistema que la gestiona, conocido como **SGBD**. La **BD** se refiere al conjunto de datos que se almacenan, mientras que el **SGBD** se encarga de administrarla, determinando su estructura, orden, permisos de acceso y dependencias.

La gestión de las **BD** ha evolucionado significativamente, convirtiéndose en el núcleo de los entornos informáticos modernos. El crecimiento del uso de Internet ha llevado a un aumento exponencial en el número de usuarios de **BD**, que a menudo interactúan con ellas sin ser plenamente conscientes de ello.

Para quienes se forman en informática, es crucial entender los **SGBD**, incluyendo sus conceptos fundamentales, diseño, lenguajes y su implementación.

3.1 Historia y necesidad del uso de las **BD**.

Para mejorar la eficiencia en el manejo de datos electrónicos, en los años 60 surgió el concepto de **BD electrónica** como una capa de software separada entre el **SO** y las aplicaciones. Esto se debió a la creciente complejidad en la manipulación de archivos y la gestión de permisos, que dificultaba el procesamiento electrónico de datos. La introducción de las **BD** electrónicas se convirtió en una innovación clave en este contexto.

Los primeros modelos desarrollados fueron las **BD** en red y jerárquicas, pero resultaron ser demasiado simples y técnicamente limitados. IBM revolucionó este panorama en los años 70 con el desarrollo del modelo relacional de **BD** (System R), lo que dio lugar a productos exitosos como *Oracle*, *SQL Server* y *DB2*.

Hasta la llegada de proyectos de código abierto en los años 2000, el mercado de software de **BD** estuvo dominado por grandes empresas. Entre los sistemas libres más populares se encuentran *MariaDB/MySQL* y *PostgreSQL*.

La tendencia hacia los sistemas **NoSQL** a partir de 2001, impulsada por la necesidad de manejar grandes volúmenes de datos no estructurados, debilitó la posición de los sistemas de **BD** de los grandes fabricantes. Surgieron soluciones como *Redis* (**BD** en memoria de estructura clave-valor), *Cassandra* (desarrollada inicialmente por *Facebook* para datos distribuidos) y *MongoDB* (**BD NoSQL** orientada a documentos).

La computación en la nube y el Big Data (2010-2019) transformaron aún más el panorama de las *BD*. Servicios como *Amazon RDS*, *Google Cloud SQL* y *Azure SQL Database* permiten gestionar *BD* sin infraestructura local.

En la actualidad (2020-presente), la tecnología de *BD* sigue evolucionando con innovaciones como las *BD* autónomas, distribuidas, en tiempo real y cuánticas.

Hoy en día, **las *BD* son esenciales en diversos campos y cualquier software empresarial se basa en robustas *BD*** con un gran número de opciones y herramientas para los administradores del sistema. La **seguridad de los datos también ha ganado importancia**, ya que las *BD* almacenan y cifran contraseñas, datos personales y divisa digital.

3.2 Conceptos y ventajas del uso de *BD*.

A finales de los setenta, las nuevas tecnologías de manejo de datos revolucionaron la gestión de la información. Los sistemas de ficheros fueron reemplazados por los ***SGBD***, que son **programas centralizados o distribuidos que facilitan la definición, estructuración y búsqueda de datos, tanto de forma interactiva como mediante lenguajes de programación**.

Una *BD* no sólo almacena los datos de la organización, sino también una descripción de estos, conocida como **metadatos**, que se guardan en el **diccionario de datos o catálogo**. Esto permite la independencia lógica y física de los datos.

Base de datos (*BD*): colección de datos lógicamente relacionados entre sí, con una definición y descripción comunes. Estos datos están estructurados de forma que minimizan la redundancia y permiten un acceso eficiente.

Ventajas del uso de *BD*:

- ✓ **Acceso múltiple:** varios usuarios o aplicaciones pueden acceder a la *BD* simultáneamente sin problemas.
- ✓ **Utilización múltiple:** cada usuario o aplicación puede tener una visión personalizada de la *BD*.
- ✓ **Flexibilidad:** se pueden establecer diferentes formas de acceder a la información.
- ✓ **Confidencialidad y seguridad:** se pueden controlar los accesos para que solo usuarios autorizados vean ciertos datos.
- ✓ **Protección contra fallos:** existen mecanismos que permiten recuperar los datos en caso de errores.
- ✓ **Independencia física:** cambios en el hardware de almacenamiento no afectan a la *BD* ni a las aplicaciones que la utilizan.
- ✓ **Independencia lógica:** modificaciones en la *BD* no afectan a las aplicaciones que la usan.
- ✓ **Redundancia controlada:** los datos se almacenan una sola vez, aunque se puede repetir información de manera controlada si es necesario.
- ✓ **Interfaz de alto nivel:** los lenguajes de alto nivel facilitan el uso de la *BD* de forma sencilla y cómoda.
- ✓ **Consulta directa:** se dispone de herramientas interactivas que permiten el acceso directo a los datos.



Autoevaluación

Una *BD* es:

- Un programa para gestionar archivos muy grandes.
- Conjunto de datos de distinto tipo relacionados entre sí, almacenados con la mínima redundancia y posibilitando el acceso a ellos eficientemente.
- El conjunto de datos de los usuarios almacenados en un único disco duro.

3.3 ¿Quién utiliza las *BD* y para qué?

Ya se ha visto lo que es una *BD* y sus características principales, pero es necesario conocer quién las usa y con qué propósito. **¿Quién utiliza las *BD*?**

Tipo	Funciones y características
El administrador	Es responsable de la creación e implementación física de la <i>BD</i> . Decide los tipos de archivos, los índices a crear, su ubicación y otros aspectos del almacenamiento de información. Además, establece las políticas de seguridad y acceso para

Tipo	Funciones y características
	minimizar problemas.
Los diseñadores	Diseñan la estructura de la BD. Identifican los datos, las relaciones entre ellos y sus restricciones. Conocen a fondo los datos y procesos que se representarán, y deben entender las reglas de negocio de la empresa. Para obtener buenos resultados, implican a todos los usuarios de la BD desde el inicio del proceso.
Los programadores de aplicaciones	Después de diseñar y construir la BD, desarrollan los programas que usarán los usuarios finales. Estos programas permiten consultar, insertar, actualizar o eliminar datos, y se crean utilizando lenguajes de programación.
Los usuarios finales	Son los clientes de la BD. El diseño, implementación y mantenimiento de la BD se realizan para cumplir con sus requisitos de gestión de información.

¿Para qué se utilizan las BD?

Enumerar todos los campos donde se utilizan las BD es complejo. A continuación, se recopilan **algunos de los ámbitos donde se emplean:**

- ✓ **Banca:** información de clientes, cuentas, transacciones, préstamos, etc.
- ✓ **Líneas aéreas:** información de clientes, horarios, vuelos, destinos, etc.
- ✓ **Universidades:** información de estudiantes, carreras, horarios, materias, etc.
- ✓ **Transacciones de tarjetas de crédito:** compras con tarjetas, generación de extractos mensuales, etc.
- ✓ **Telecomunicaciones:** registros de llamadas, generación de facturas mensuales, mantenimiento de saldo en tarjetas telefónicas de prepago, almacenamiento de información sobre redes, etc.
- ✓ **Medicina:** información hospitalaria, biomedicina, genética, etc.
- ✓ **Justicia y seguridad:** delincuentes, casos, sentencias, investigaciones, etc.
- ✓ **Legislación:** normativa, registros, etc.
- ✓ **Organismos públicos:** datos ciudadanos, certificados, etc.
- ✓ **Sistemas de posicionamiento geográfico.**
- ✓ **Hostelería y turismo:** reservas de hotel, vuelos, excursiones, etc.
- ✓ **Ocio digital:** juegos online, apuestas, etc.
- ✓ **Cultura:** gestión de bibliotecas, museos virtuales, etc.

3.4 Ubicación de la información.

Las BD pueden variar en tamaño, desde muy pequeñas (1 MByte o menos) hasta extremadamente grandes (varios TBytes). Sin embargo, **generalmente se almacenan en discos duros** y otros dispositivos de almacenamiento, accesibles a través de un ordenador. **Una gran BD puede requerir servidores en distintas ubicaciones**, mientras que una **pequeña BD puede residir en el disco duro de un solo equipo.**

A continuación, se presentan los **sistemas de almacenamiento de información más utilizados para el despliegue de BD**, comenzando con aquellos que pueden alojar BD de tamaño pequeño y mediano, y luego analizando los sistemas de alta disponibilidad para grandes servidores.

3.4.1 Discos.

Uno de los principales medios para almacenar información son los discos. Actualmente, existen **dos tipos** principales:

- ✓ **Discos duros (HDD):** son **dispositivos mecánicos que utilizan el magnetismo para grabar datos.** Están compuestos por varios discos rígidos que giran a gran velocidad, con un cabezal de lectura/escritura que graba o lee la información.
- ✓ **Discos de estado sólido (SSD):** **almacenan la información en chips con memorias flash** interconectadas (memorias NAND). Se diferencian en tres tipos según su conexión: SATA, M.2 y PCIe NVME.



3.4.2 Cintas magnéticas.

Las cintas magnéticas son dispositivos de almacenamiento que **graban datos en pistas sobre una banda plástica recubierta de material magnético.** Existen **diferentes tipos de cintas** según su composición química y los formatos de grabación que utilizan.

Aunque es un **sistema de almacenamiento antiguo**, sigue siendo útil en el entorno actual de manejo de datos masivos. Su **gran durabilidad** y **bajo costo** las hacen ideales **para copias de seguridad**, por lo que aún se utilizan hoy en día.

3.4.3 RAID.

RAID (**Redundant Array of Independent Disks**) es una tecnología de almacenamiento redundante que utiliza **dos o más discos duros** formando una **unidad de trabajo** para **mejorar la capacidad, velocidad y seguridad** del almacenamiento.

Tipos de RAID:

- ✓ **RAID 0:** los **datos se distribuyen de forma alterna entre los discos**, aumentando significativamente la **velocidad de acceso**. Sin embargo, **no ofrece redundancia**; si un disco falla, se pierden todos los datos.
- ✓ **RAID 1:** requiere **al menos dos discos duros**, y los **datos se duplican** en cada uno de ellos. Esto proporciona **alta redundancia**, ya que, si un disco falla, los datos siguen disponibles en el otro. La **capacidad total es igual a la del disco de menor tamaño**.
- ✓ **RAID 5:** necesita un **mínimo de tres discos**. Los **datos y la información de paridad se distribuyen entre los discos**, permitiendo reconstruir los datos en caso de fallo de un disco. Si un disco falla, los datos pueden recuperarse utilizando la información de paridad de los otros discos.
- ✓ **Otros tipos de RAID:**
 - **RAID 6:** similar a **RAID 5**, pero con una **duplicación adicional de paridad en dos discos**, proporcionando mayor tolerancia a fallos.
 - **RAID 0+1 (RAID 01):** requiere **al menos cuatro discos**. Combina **dos matrices RAID 0 en una matriz RAID 1**, ofreciendo **alta velocidad y duplicación de datos**.
 - **RAID 1+0 (RAID 10):** inverso a **RAID 0+1**. Combina **dos matrices RAID 1 en una matriz RAID 0**, ofreciendo redundancia y velocidad. También requiere al menos cuatro discos.
 - **RAID 5+0 (RAID 50):** necesita **al menos seis discos**, formando **dos matrices RAID 5, que luego se combinan en una matriz RAID 0**, proporcionando las ventajas de ambos tipos.

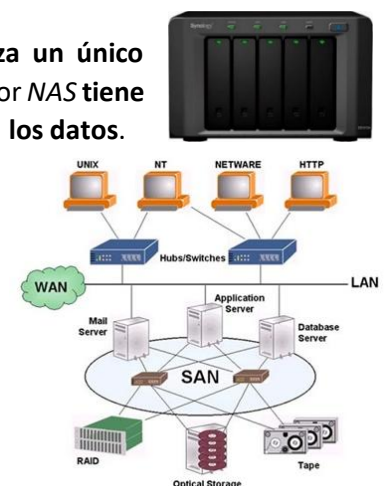
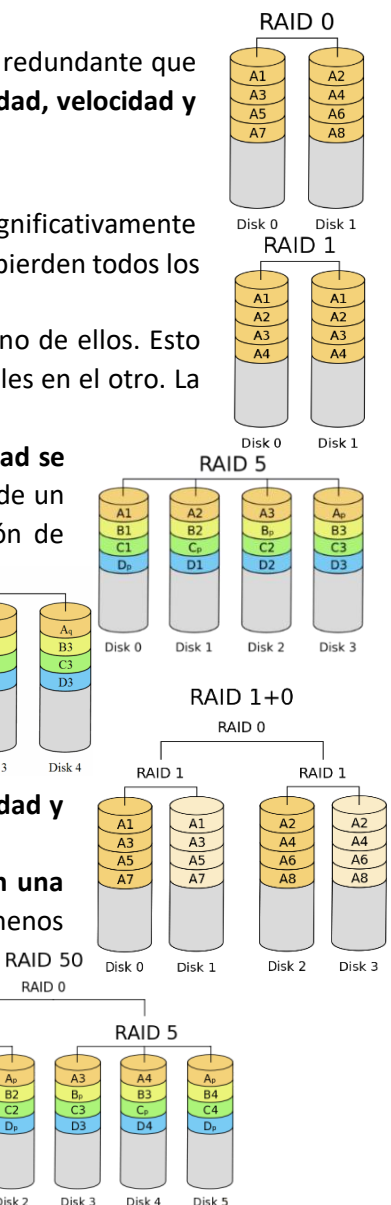
Estos sistemas permiten configuraciones flexibles y escalables según las necesidades de capacidad, velocidad y redundancia.

3.4.4 Almacenamiento en red.

Las redes han evolucionado con el paso del tiempo y las actuales redes empresariales tienen una capacidad de transferencia de al menos **1000 Mbps** (o **10 Gbps** en redes de fibra óptica), **permitiendo transferir grandes cantidades de información** rápidamente. Esto ha popularizado **métodos de almacenamiento en red** como **NAS** y **SAN**, usados principalmente en entornos empresariales:

- ✓ **NAS** (**Network Attached Storage**): es un **sistema de almacenamiento que utiliza un único dispositivo accesible desde la red para guardar y compartir información**. Un servidor NAS tiene su propio hardware y **SO** que gestiona el acceso de usuarios y las operaciones con los datos.
- ✓ **SAN** (**Storage Area Network**): utilizado principal en **servidores de aplicaciones**, es una **solución de almacenamiento en red donde múltiples dispositivos actúan como bloques de disco**, permitiendo el **almacenamiento y acceso a la información desde cualquier punto de la red**.

Estos sistemas permiten que los recursos de almacenamiento estén disponibles para varios servidores en una red local (**LAN**) o amplia (**WAN**). Al no residir la información almacenada directamente en ningún servidor, se optimiza la capacidad de procesamiento para aplicaciones comerciales y se puede proporcionar almacenamiento adicional al servidor que lo necesite.



3.4.5 Almacenamiento en la nube.

Hoy en día, los sistemas de almacenamiento de datos tienden a migrar al **almacenamiento en la nube**. Esto consiste en contratar **espacio** en los servidores de un proveedor de almacenamiento en la nube, **permitiendo almacenar y acceder a la información** desde cualquier **SO**, ubicación, hora o dispositivo.

El aumento de la tasa de transmisión de datos por Internet ha facilitado el acceso a los servicios de almacenamiento en la nube. Actualmente, es posible contratar suscripciones a estos servicios con grandes capacidades de almacenamiento a costos muy reducidos para las empresas.

Las **ventajas** del almacenamiento en la nube en términos de **seguridad, portabilidad y accesibilidad** lo han convertido en el sistema de almacenamiento más utilizado por las empresas. Sin embargo, el almacenamiento en la nube **requiere una conexión constante a Internet**, por lo que es recomendable combinarlo con otros tipos de almacenamiento para garantizar el acceso a la información en todo momento.



Autoevaluación

Rellena los huecos con los conceptos adecuados.

Un tipo de red donde se optimiza el poder de procesamiento para aplicaciones comerciales, pudiendo proporcionarse la capacidad de almacenamiento en el servidor donde más se necesite, se denomina sistema _____.



TAREA

- ¿Cuáles son las ventajas de las **BD** en comparación con los sistemas de gestión de datos basados en archivos?
- Según la ubicación de la información, ¿Qué tipos de almacenamiento se pueden tener? ¿qué ventajas e inconvenientes presenta cada uno de ellos?

4 Evolución de los modelos de **BD**.

Un modelo de datos es una representación abstracta que define cómo se estructuran los datos, las relaciones entre ellos y las restricciones que se aplican.

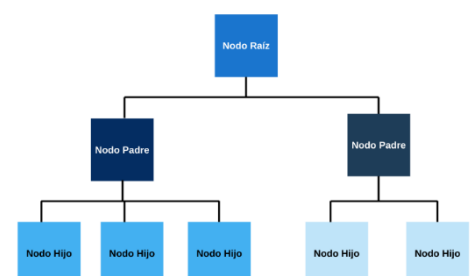
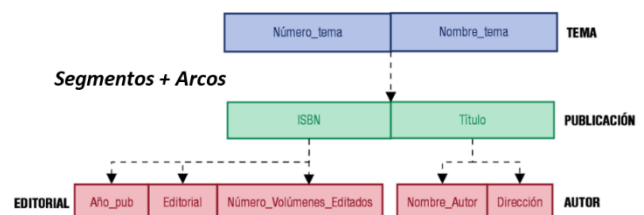
4.1 Modelo jerárquico.

El modelo jerárquico, también conocido como **modelo en árbol**, apareció a principios de los años sesenta. **Utiliza una estructura de árbol invertido para organizar los datos**, donde estos se almacenan en **segmentos** que se relacionan mediante **arcos**.

La **información** en este modelo **se organiza jerárquicamente en relaciones de tipo padre/hijo**. Los nodos contienen atributos o campos y se conectan con nodos hijos. Un nodo puede tener múltiples hijos, pero solo un padre. Debido a sus limitaciones, el modelo jerárquico está **en desuso** hoy en día.

Características principales de las **BD** jerárquicas:

- ✓ **Estructura en árbol invertido**: con un nodo raíz, nodos padre y nodos hijos.
- ✓ **Niveles jerárquicos**: el nodo raíz está en el nivel 0.
- ✓ **Arcos (enlaces)**: representan las **asociaciones jerárquicas entre nodos** y no tienen nombre, ya que solo puede existir una interrelación entre dos conjuntos de datos.
- ✓ **Nodos hijos y padres**: un nodo **padre** puede tener **múltiples nodos hijos**, pero un **nodo hijo** solo puede tener un padre.
- ✓ **Nodo raíz**: es el **único nodo que no tiene padre**.



- ✓ **Nodos hoja:** son los **nodos sin descendientes**.
- ✓ **Altura:** se refiere a los **niveles de la estructura jerárquica**.
- ✓ **Momento:** se refiere al **número de nodos en el árbol**.
- ✓ **Orden fijo:** el árbol se recorre en un orden preestablecido.
- ✓ **Relaciones:** solo pueden ser **de uno a uno o de uno a varios**.
- ✓ **Estructura inmutable:** una vez establecida, la estructura del árbol no se puede modificar.

4.2 Modelo en red.

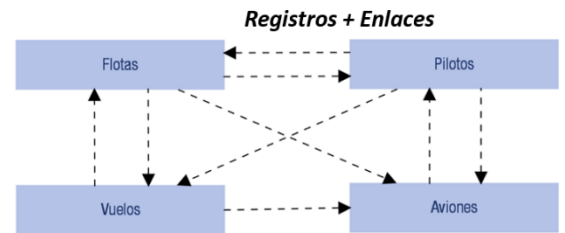
El modelo de datos en red **apareció a mediados de los sesenta como respuesta a las limitaciones del modelo jerárquico para representar relaciones más complejas**. Está **en desuso** hoy en día.

Organiza la información en registros y enlaces. Los registros almacenan los datos, mientras que los enlaces relacionan estos datos. A diferencia del modelo jerárquico, en el modelo en red **un nodo puede tener más de un padre**.

Este modelo **permite representar cualquier tipo de relación entre los datos** sin duplicar información, lo que ahorra espacio de almacenamiento. Sin embargo, **su manejo puede ser complicado**. El sistema de gestión de información más popular basado en este modelo es el sistema *IDMS*.

Ventajas del modelo en red:

- ✓ **Relaciones de muchos a muchos:** permite establecer relaciones complejas, como en una tienda online que relaciona productos con pedidos (no es posible en el modelo jerárquico).
- ✓ **Eficiencia en el uso del espacio:** al no duplicar información, se optimiza el almacenamiento.



4.3 Modelo relacional.

El modelo relacional, **desarrollado por E.F. Codd en 1970**, es actualmente el **más utilizado para BD**. En este modelo, la **BD se percibe por el usuario como un conjunto de tablas**, aunque a nivel físico puede estar implementada de diferentes maneras.

En el modelo relacional, los **datos y sus relaciones se representan lógicamente mediante tablas bidimensionales** (relaciones). Cada **fila de la tabla** se denomina **registro o tupla**, y cada **columna** se llama **campo o atributo**. El **conjunto de valores posibles para un atributo** se conoce como **dominio**.

Una **clave primaria** es un **atributo o conjunto de atributos que identifica de forma única a cada registro**.

Requisitos para las tablas:

- ✓ **Nombre único:** cada tabla debe tener un nombre único.
- ✓ **Uniformidad de registros:** todos los registros deben ser del mismo tipo.
- ✓ **Atributos únicos:** no debe haber campos o atributos repetidos.
- ✓ **Registros únicos:** no deben existir registros duplicados.
- ✓ **Sin orden específico:** no hay un orden fijo para el almacenamiento de los registros.

El **lenguaje** utilizado para trabajar con las **BD relacionales** es **SQL** (*Structured Query Language*), un **estándar implementado por los principales SGBD relacionales**.

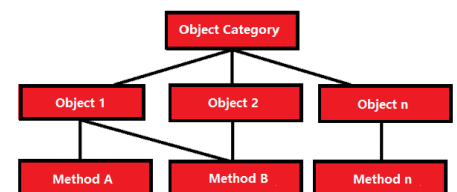


4.4 Modelo orientado a objetos.

El modelo orientado a objetos **define la BD en términos de objetos, sus propiedades y sus operaciones**. Los **objetos con la misma estructura y comportamiento pertenecen a una clase**, y estas clases se organizan en **jerarquías**.

El **objetivo** del modelo orientado a objetos es **superar las limitaciones del modelo relacional**. Introduce mejoras como la **herencia entre tablas y los tipos definidos por el usuario**.

Los **conceptos clave del paradigma de objetos** que el modelo orientado a objetos incorpora son:



- ✓ **Encapsulación:** permite **ocultar la información a otros objetos**, previniendo accesos incorrectos o conflictos.
- ✓ **Herencia:** los **objetos heredan comportamientos** (propiedades y operaciones) dentro de una jerarquía de clases.
- ✓ **Polimorfismo:** una **operación puede aplicarse a distintos tipos de objetos**.

Desde la aparición de la *Programación Orientada a Objetos (POO)*, se consideró la adaptación de las *BD* a estos lenguajes. Sin embargo, **no han reemplazado a las *BD* relacionales**.

4.5 Modelo orientado a documentos.

En el modelo orientado a documentos, **los datos se almacenan como documentos**, que son **estructuras semi-estructuradas generalmente en formato *JSON***. Estos **documentos se agrupan en colecciones**, y cada documento **contiene pares clave-valor**, donde **cada clave es única dentro del documento**.

Una característica importante de este modelo es que **no requiere un esquema fijo**, lo que **permite que los documentos en una misma colección puedan tener diferentes estructuras**. Esto facilita la evolución del modelo de datos sin necesidad de alterar las estructuras existentes.

Los **documentos pueden contener otros documentos o arrays**, lo que permite representar relaciones jerárquicas complejas de manera más natural. Cada **documento es una unidad cerrada en sí misma**, y **no es necesario establecer relaciones entre documentos como en otros modelos**.

En comparación con el modelo relacional, donde varias tablas se conectan para seleccionar un registro común, en el modelo documental un único documento puede contener toda la información necesaria. Esto simplifica el almacenamiento y la recuperación de datos.

Las *BD* documentales son **especialmente útiles para aplicaciones web**, ya que permiten almacenar formularios *HTML* completos. Con el avance de la *Web 2.0*, estas *BD* han ganado popularidad. Es importante destacar que **existen diferencias significativas entre los diversos sistemas basados en documentos**, desde la sintaxis hasta la estructura interna, por lo que no todas son apropiadas para cualquier escenario.

Key	First Name	Surname	Zipcode ID
1	Walker	McClain	1
2	Blain	Muller	2
3	Jack	Schmidt	2
4	Greg	Cohn	4

Zipcode ID	City	State	Zipcode
1	Bend	Oregon	97701
2	Niwot	Colorado	80503
3	Spartanburg	South Carolina	29301
4	Syracuse	New York	13201


```

{
  "ID": 3,
  "First Name": "Jack",
  "Surname": "Schmidt",
  "Zipcode": "80503",
  "City": "Niwot",
  "State": "Colorado"
}
  
```

JSON

Ejemplos de sistemas basados en documentos: *Lotus Notes*, *Amazon SimpleDB*, *MongoDB*, *CouchDB* y *OrientDB*.

4.6 Otros modelos.

4.6.1 Modelo objeto-relacional.

Las *BD* pertenecientes al modelo objeto-relacional **son un híbrido entre las *BD* del modelo relacional y el orientado a objetos**. El principal inconveniente de las *BD* orientadas a objetos radica en los costos de la conversión de las *BD* relacionales a *BD* orientadas a objetos.

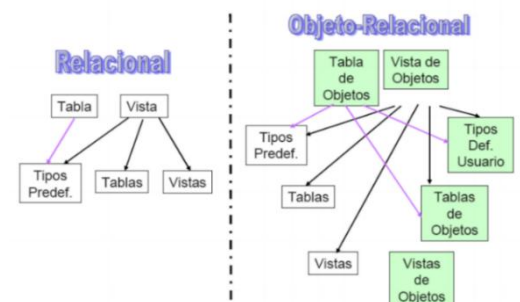
En una *BD* objeto-relacional (*BDOR*), **se busca obtener lo mejor del modelo relacional, incorporando las mejoras ofrecidas por la orientación a objetos**. En este modelo, se siguen almacenando tuplas, pero la estructura de las tuplas no está restringida, y las relaciones pueden ser definidas en función de otras, lo que se denomina herencia directa.

El **estándar en el que se basa este modelo es *SQL99***. Este estándar ofrece la posibilidad de añadir a las *BD* relacionales **procedimientos almacenados, triggers, tipos definidos por el usuario, consultas recursivas, *BD* OLAP, tipos *LOB* (*BLOB* y *CLOB*), etc.**

La mayoría de las *BD* relacionales de gran tamaño, como *Oracle*, *PostgreSQL*, *SQL Server*, entre otras, son objeto-relacionales.

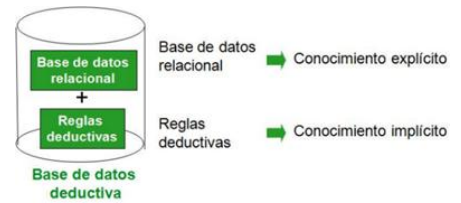
4.6.2 Modelo de *BD* deductivas o lógicas.

En el modelo de *BD* deductivas o lógicas, **las *BD* almacenan información y permiten realizar deducciones a través**



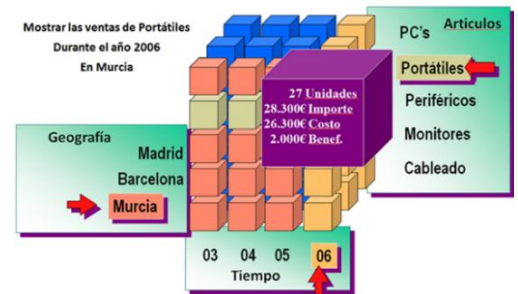
de inferencias. Esto significa que se derivan nuevas informaciones a partir de las que se han introducido explícitamente en la *BD*.

Estas *BD* son también llamadas *BD lógicas*, ya que se basan en la lógica matemática. Surgieron para contrarrestar las limitaciones del modelo relacional en cuanto a la respuesta a consultas recursivas y la deducción de relaciones indirectas entre los datos almacenados.



4.6.3 *BD* multidimensionales.

Las *BD* multidimensionales están diseñadas para aplicaciones específicas. En estas *BD*, la información se representa como matrices multidimensionales, cuadros de múltiples entradas o funciones de varias variables sobre conjuntos finitos. Cada una de estas matrices se denomina cubo.



4.6.4 *BD* transaccionales.

Las *BD* transaccionales se caracterizan por su velocidad para gestionar el intercambio de información. Se utilizan principalmente en sistemas bancarios, análisis de calidad y datos de producción industrial. Estas *BD* son altamente fiables, ya que cada una de las operaciones de inserción, actualización o borrado se realizan completamente o se descartan.

4.7 Tabla comparativa de los principales modelos.

Modelo de BD	Desarrollo	Ventajas	Inconvenientes	Ámbitos de aplicación	Marcas
Jerárquico (segmentos y arcos)	Década de 1960	Acceso de lectura muy rápido, estructura clara, técnicamente simple.	Estructura fija en árbol que no permite conexiones entre árboles.	Bancos, compañías de seguros, sistemas operativos.	IMS/DB
En red (registros y enlaces)	Mediados década de 1960	Admite varias formas de acceder a un registro, sin jerarquía estricta.	En <i>BD</i> más grandes no se tiene una vista general.	Grandes ordenadores.	UDS (Siemens), DMS (Sperry Univac)
Relacional (tablas)	1970	Simple, flexible. Creación y edición fácil y flexible, fácil de ampliar, rápida puesta en marcha, contexto de competencia muy dinámica.	Inmanejable con cantidades grandes de datos, segmentación deficiente, atributos de clave artificiales, interfaz de programación externa, no refleja bien las propiedades y la conducta de los objetos.	Control de gestión, facturación, sistemas de control de inventario, sistemas de gestión de contenido, etc.	MySQL, PostgreSQL, Oracle, SQLite, DB2, Ingres, MariaDB, Microsoft Access
Orientado a objetos (objetos)	Final de la década de 1980	Soporta mejor los lenguajes de programación orientados a objetos, permite almacenar contenido multimedia.	El rendimiento empeora con grandes volúmenes de datos, pocas interfaces compatibles.	Inventario (museos, comercio minorista).	db4o
Orientado a documentos (documentos sin esquema definido)	Década de 1980	Los datos relacionados se guardan de forma centralizada en documentos independientes, estructura libre, concepción multimedia.	El trabajo de organización es relativamente alto, a menudo requiere conocimientos de programación.	Aplicaciones web, buscadores, <i>BD</i> de texto.	Lotus Notes, Amazon SimpleDB, MongoDB, CouchDB, Riak, ThruDB, OrientDB



TAREA

- Teniendo en cuenta los distintos modelos de *BD*, ¿cuál es el más utilizado en la actualidad? ¿cuáles son los modelos de *BD* que han quedado obsoletos? ¿qué tipos de *BD NoSQL* están emergiendo actualmente y en qué contextos suelen aplicarse?
- Visitar la URL <https://db-engines.com/en/ranking> e identificar el modelo de *BD* más utilizado dentro de las 10 primeras posiciones. ¿Qué otros modelos de *BD* (que no sean relacionales) se encuentran en las primeras 10 posiciones? Investigar y describir los modelos presentes en las primeras 10 posiciones.

5 Sistemas Gestores de Bases de Datos (SGBD).

Los Sistemas Gestores de Bases de Datos (SGBD), también conocidos como DBMS (DataBase Management System), son conjuntos coordinados de programas, procedimientos, lenguajes, etc., que permiten acceder y gestionar los datos almacenados en las *BD*. Su objetivo principal es proporcionar eficiencia y seguridad en la recuperación y

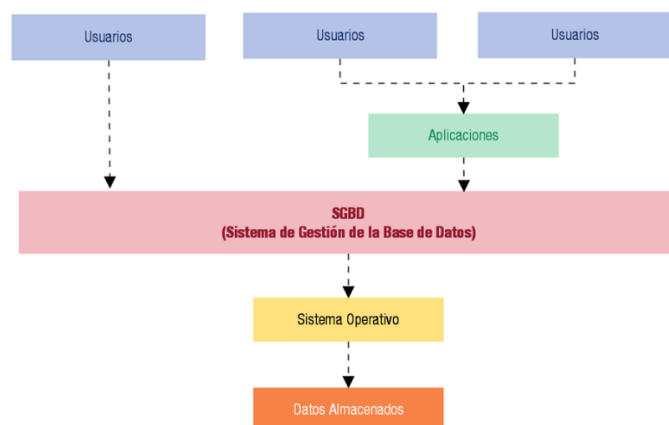
actualización de información en las BD, especialmente diseñados para manejar grandes volúmenes de datos.

El **SGBD permite crear y mantener BD**, facilitando su definición (especificación de tipos de datos, estructuras y restricciones), construcción (almacenamiento concreto de los datos en algún soporte) y manipulación de la información contenida en ellas (realización de consultas para recuperar información o actualizar los datos).

Las **ventajas del uso de SGBD** son numerosas:

- ✓ **Proporcionan una visión abstracta de los datos, ocultando la complejidad relacionada con su almacenamiento y mantenimiento.**
- ✓ **Ofrecen independencia física y lógica**, permitiendo que la visión y manipulación de los datos sean independientes de cómo estén almacenados físicamente.
- ✓ **Reducen la redundancia y la inconsistencia de los datos.**
- ✓ **Aseguran la integridad de los datos.**
- ✓ **Facilitan el acceso a los datos**, proporcionando rapidez y evitando la pérdida de información.
- ✓ **Aumentan la seguridad y privacidad de los datos.**
- ✓ **Mejoran la eficiencia en la gestión de datos.**
- ✓ **Permiten compartir datos y accesos concurrentes.**
- ✓ **Facilitan el intercambio de datos** entre diferentes sistemas.
- ✓ **Incorporan mecanismos de copia de seguridad y recuperación** para restablecer la información en caso de fallos en el sistema.

Los **SGBD** interactúan con otros elementos de software en el sistema, especialmente con el **SO**. Los datos almacenados en la **BD** son utilizados por otras aplicaciones, y el **SGBD** proporciona facilidades para el acceso y manipulación de la información, basándose en las funciones y métodos del **SO**.



5.1 Servicios y funciones proporcionados por un SGBD.

Los **servicios principales** que un **SGBD** debe prestar son los siguientes:

- ✓ **Creación y definición de la BD:** permite especificar la estructura de la **BD**, incluyendo el tipo de datos, restricciones y relaciones entre ellos. Esta información se almacena en el diccionario de datos.
- ✓ **Manipulación de los datos:** permite realizar consultas para recuperar información, inserciones para añadir nuevos datos y actualizaciones para modificar los datos existentes. Esto se realiza mediante lenguajes de manipulación de datos.
- ✓ **Control de acceso a los datos:** proporciona mecanismos de seguridad para controlar quién puede acceder y modificar los datos en la **BD**. Esto incluye autenticación de usuarios, autorización de permisos y gestión de roles.
- ✓ **Mantenimiento de la integridad y consistencia de los datos:** utiliza mecanismos para garantizar que los datos sean precisos y coherentes. Esto incluye la aplicación de restricciones de integridad, como claves primarias y foráneas, y la realización de validaciones de datos.
- ✓ **Acceso compartido a la BD:** gestiona la interacción entre múltiples usuarios que acceden a la **BD** al mismo tiempo, garantizando la consistencia de los datos y evitando conflictos.
- ✓ **Mecanismos de respaldo y recuperación:** proporciona funciones para realizar copias de seguridad periódicas de la **BD** y recuperar los datos en caso de fallos del sistema, errores humanos o desastres naturales.

Las **tres funciones fundamentales** de un **SGBD** son:

1. **Función de descripción o definición:** permite al diseñador de la **BD** crear las estructuras apropiadas para integrar los datos de manera efectiva. Esta función se realiza mediante el **Lenguaje de Descripción de Datos (DDL)**, que define las estructuras, relaciones y reglas que deben seguir los datos en **tres niveles: interno, conceptual y externo**. El **DDL** especifica características como el espacio en disco reservado, la longitud de los campos, las entidades y atributos, las interrelaciones y las vistas de los usuarios.
2. **Función de manipulación:** permite a los usuarios buscar, agregar, eliminar o modificar los datos de la **BD** de

acuerdo con las especificaciones y normas de seguridad establecidas por el administrador. Esta función se realiza mediante el **Lenguaje de Manipulación de Datos (DML)**, que proporciona herramientas para realizar consultas, inserciones, eliminaciones y modificaciones de datos almacenados en la BD.

3. **Función de control:** permite al administrador **establecer mecanismos de protección para las diferentes visiones de los datos asociadas a cada usuario**, así como realizar **tareas de administración** como la creación de copias de seguridad, la carga de archivos, la auditoría y la configuración del sistema. Esta función se realiza mediante el **Lenguaje de Control de Datos (DCL)**, que proporciona sentencias para la gestión de usuarios, permisos y seguridad de la BD.

Estas funciones se desarrollan usando el **Lenguaje de Consulta Estructurado (SQL)** en los **SGBD relacionales**, que **proporciona sentencias para operaciones de DDL, DML y DCL**. Además, los **SGBD** suelen ofrecer otras herramientas complementarias como generadores de formularios, informes, interfaces gráficas y generadores de aplicaciones.



Autoevaluación

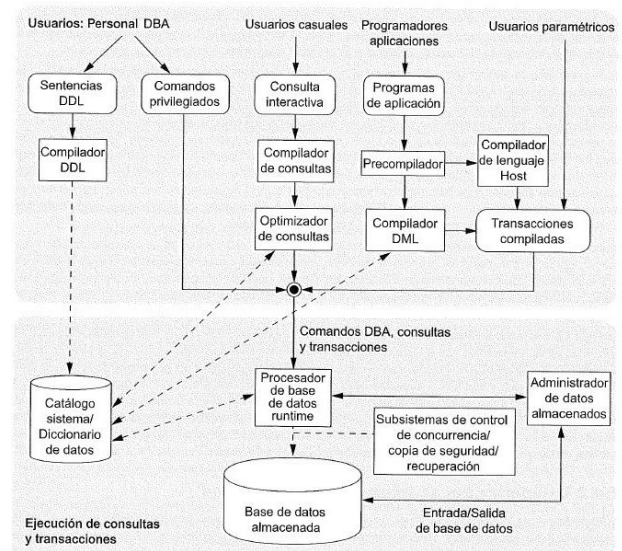
El **DDL** sirve para:

- La introducción de los datos en una BD.
- Definir la estructura lógica de la BD.
- Interrogar a la BD (consultar la información de dicha BD).

5.2 Componentes de un SGBD.

Un **SGBD** es un paquete de software complejo que ha de proporcionar servicios relacionados con el **almacenamiento y la explotación de los datos** de forma eficiente. Para ello, cuenta con una serie de **componentes** que se detallan a continuación:

1. **Lenguajes de la BD:** estos lenguajes **permiten a los usuarios interactuar con la BD**. Incluyen el **DDL** para **definir la estructura de la BD**, el **DML** para **realizar consultas y modificaciones en los datos** y el **DCL** para **controlar el acceso y la seguridad de la BD**.
2. **Diccionario de datos:** es una **colección de metadatos que describe los datos almacenados en la BD**. Contiene información sobre la **estructura lógica y física de la BD**, **definición de tablas, vistas, índices, restricciones de integridad, permisos de usuario, auditoría de acceso**, etc.
3. **Gestor de BD (SGBD):** es el **componente central del sistema que gestiona y controla todas las operaciones relacionadas con la BD**. Proporciona una **interfaz entre los datos almacenados y los programas de aplicación que acceden a ellos**. Garantiza la privacidad, seguridad e integridad de los datos, controla los accesos concurrentes y se comunica con el sistema operativo.
4. **Usuarios de la BD:** los **SGBD** admiten **diferentes perfiles de usuario, cada uno con diferentes permisos y privilegios sobre los objetos de la BD**. Esto incluye al **administrador de la BD (DBA)**, que tiene el control centralizado de la BD, así como a los **operadores, analistas, programadores, diseñadores y usuarios finales** que interactúan con los datos de diversas formas.
5. **Herramientas de la BD:** estas son **aplicaciones adicionales que facilitan la gestión de la BD y mejoran la productividad de los usuarios**. Pueden incluir herramientas para administrar la BD, gestionar usuarios y permisos, generar formularios e informes, desarrollar interfaces gráficas y crear aplicaciones basadas en la BD.



5.3 Arquitectura ANSI/SPARC.

La arquitectura **ANSI/SPARC** es una **estructura de referencia (estándar)** para los **SGBD** que ayuda a organizar y gestionar la **BD** de manera eficiente. Los **tres niveles de abstracción** que proporciona son:

- ✓ **Nivel interno o físico** (perspectiva del almacenamiento físico): este nivel **se enfoca en la estructura física de**

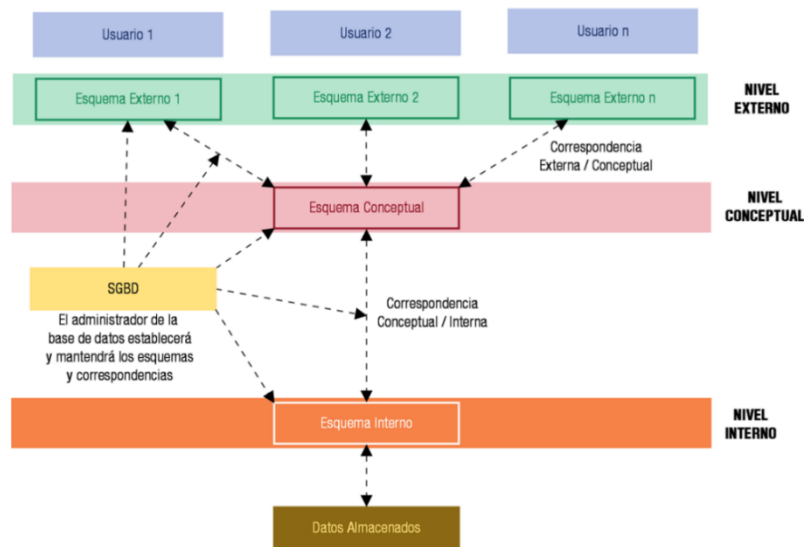
la **BD**. Aquí se **describe cómo se almacenan los datos en el sistema de almacenamiento a través del esquema interno**. Define las estructuras de almacenamiento, como los índices, bloques, y la ubicación física de los datos en disco. Está **gestionado por el propio SGBD**, los usuarios no interactúan directamente con este nivel.

- ✓ **Nivel lógico o conceptual** (perspectiva del administrador): en este nivel, **se describe la estructura completa de la BD de manera lógica a través del esquema conceptual**. Se detallan las **entidades, atributos, relaciones, operaciones y restricciones** de la **BD**. Se centra en qué datos se almacenan y su estructura lógica, independientemente de la organización física.
- ✓ **Nivel externo o de visión del usuario** (perspectiva del usuario/aplicación): en este nivel **se describen las diferentes vistas que los usuarios/aplicaciones tendrán de la BD a través de los esquemas externos**. Cada usuario o grupo de usuarios verá solo la parte de la **BD** que le concierne, ocultando el resto. Proporciona control de acceso y personalización en función del tipo de usuario.

Para una **BD**, sólo existirá un único esquema interno, un único esquema conceptual y podrían existir varios esquemas externos definidos para uno o varios usuarios.

La arquitectura **ANSI/SPARC** permite que cada grupo de usuarios haga referencia a su propio esquema externo, y el **SGBD** se encarga de transformar cualquier solicitud de usuario en términos del esquema conceptual y luego al esquema interno para procesarla sobre la **BD** almacenada. Este proceso de transformación **garantiza la independencia de datos a dos niveles**:

- ✓ **Independencia lógica**: permite **modificar el esquema conceptual sin afectar los esquemas externos ni los programas de aplicación**.
- ✓ **Independencia física**: permite **modificar el esquema interno sin necesidad de alterar el conceptual o el externo**, lo que facilita los cambios en el sistema de almacenamiento sin afectar a otros niveles del sistema.



Autoevaluación

El esquema conceptual de la totalidad de la **BD** puede obtenerse de la unión de todos los esquemas externos definidos para cada usuario de la **BD**. ¿Verdadero o falso? Razona la respuesta.

5.4 Tipos de SGBD.

Los **SGBD** se pueden clasificar según diferentes **criterios**:

- a) **Modelo lógico**. → Ver punto 4 de la unidad.
- b) **Número de usuarios**:
 - ✓ **Monousuario**: atienden a un usuario a la vez, común en ordenadores personales.
 - ✓ **Multiusuario**: atienden a múltiples usuarios al mismo tiempo, la mayoría de los **SGBD** se encuentran en esta categoría.
- c) **Distribución de la BD**:
 - ✓ **Centralizados**: los datos se almacenan en un solo ordenador (**SGBD** y **BD** residen en una sola máquina).
 - ✓ **Distribuidos**: la **BD** y el **SGBD** se distribuyen en varios sitios conectados por una red. Pueden ser **homogéneos** (mismo **SGBD** en múltiples sitios) o **heterogéneos** (diferentes **SGBD** en múltiples sitios).
- d) **Costo**. Varía según la funcionalidad y el proveedor. Los precios pueden oscilar desde paquetes gratuitos hasta costosos sistemas de propósito especial.
- e) **Propósito**:
 - ✓ **Propósito general**: **SGBD** diseñados para el tratamiento de cualquier tipo de **BD** y aplicación.
 - ✓ **Propósito específico**: diseñados y contruidos para una aplicación específica donde el rendimiento es crucial. No son adecuados para otras aplicaciones.



TAREA

8. ¿Cuáles son las funciones principales de un **SGBD**? Describir detalladamente cada una de las funciones.
9. ¿Cuáles son los componentes fundamentales de un **SGBD**? Explicar la función de cada uno de estos componentes.
10. ¿Cuántos esquemas conceptuales y esquemas externos puede tener una **BD**? ¿cuál es la diferencia entre ellos?
11. (Opcional) En base a la imagen proporcionada, describir los pasos que sigue un **SGBD** para interpretar una solicitud de datos de un usuario específico.



6 SGBD comerciales y libres.

En el mercado actual de software, hay una **gran variedad de SGBD**, tanto comerciales como de código abierto. **Elegir el adecuado para un proyecto es una decisión crucial**, ya que el más avanzado no siempre es el mejor para cada situación. Aspectos como el volumen de carga, el **SO** utilizado, el presupuesto y los plazos de entrega deben considerarse cuidadosamente.

Los **SGBD comerciales** más utilizados son:

- ✓ **Oracle**: reconocido globalmente, es confiable y seguro, basado en un modelo de datos relacional. Aunque es potente, su precio alto lo hace más común en empresas grandes.
- ✓ **MySQL Standard/Enterprise/Clúster**: ampliamente extendido. Es relacional, multihilo, multiusuario y multiplataforma, ideal para consultas de **BD** y plataformas web.
- ✓ **DB2**: multiplataforma, integra **XML** nativamente con *pureXML* de **IBM**, permitiendo operaciones jerárquicas en documentos **XML**.
- ✓ **Informix**: otra opción de **IBM**, ofrece un **SGBD** relacional sencillo y confiable, con utilidades avanzadas para conectividad y tecnologías web.
- ✓ **Microsoft SQL SERVER**: relacional, exclusivo para **Windows** y con arquitectura cliente/servidor, compitiendo con otras opciones como **Oracle** y **MySQL**.

Además, existen **otros SGBD comerciales** importantes como **dBase, Access, Interbase y FoxPro**.

Por otro lado, los **SGBD de código abierto**, también llamados *Open Source*, son alternativas populares. Algunos de los más utilizados son:

- ✓ **MySQL Community**: relacional, multihilo y multiusuario, multiplataforma, con millones de instalaciones. Accesible a través de múltiples lenguajes de programación y muy ligado a aplicaciones web.
- ✓ **MariaDB**: derivado de **MySQL**, con alta compatibilidad y licencia **GPL**.
- ✓ **PostgreSQL**: relacional y Orientado a Objetos, considerado el **SGBD** más avanzado de código abierto, multiplataforma y accesible desde varios lenguajes de programación.
- ✓ **Firebird**: relacional, multiplataforma y con bajo consumo de recursos, basado en **Interbase**.
- ✓ **Apache Derby**: escrito en **Java**, con soporte multilenguaje y multiplataforma.
- ✓ **SQLite**: relacional, escrito en **C**, con tiempos de acceso rápidos y soporte para múltiples lenguajes.

Cada uno de estos **SGBD** tiene sus propias fortalezas y debilidades, por lo que es importante evaluar cuidadosamente las necesidades del proyecto antes de tomar una decisión.

7 BD centralizadas.

Sistema de BD centralizado: el **SGBD** está implantado en una única plataforma u ordenador, desde donde se gestionan todos los recursos de manera centralizada. Esta arquitectura es típica de los centros de procesamiento de datos tradicionales, basándose en tecnologías sencillas, robustas y bien experimentadas.

La arquitectura de un sistema de *BD* depende del sistema informático que la sustenta. Tradicionalmente, la arquitectura centralizada **fue la más común, aunque hoy en día se utiliza menos**.

En los años sesenta y setenta, los sistemas eran totalmente centralizados, acorde a los *SO* y hardware de la época: un gran ordenador para toda la empresa y una red de terminales sin inteligencia ni memoria.

Características principales de las *BD* centralizadas:

- ✓ **Almacenamiento único:** toda la *BD* se encuentra en una ubicación central.
- ✓ **Sin elementos de procesamiento distribuidos:** a diferencia de las *BD* distribuidas, no tiene múltiples elementos de procesamiento ni mecanismos de intercomunicación.
- ✓ **Componentes:** los datos, el *SGBD* y los dispositivos de almacenamiento secundario están todos en un solo lugar.
- ✓ **Seguridad:** puede ser más fácilmente comprometida debido a su centralización.



Ventajas e inconvenientes de las *BD* centralizadas:

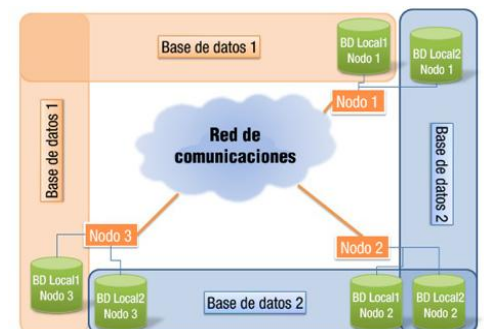
Ventajas	Inconvenientes
Evita la redundancia: previene inconsistencias y desperdicio de espacio.	Menor poder de cómputo: un mainframe no tiene más poder de cómputo comparado con un sistema distribuido.
Evita la inconsistencia: la no-concordancia de datos no puede ocurrir si se guarda en un solo sitio.	Fallas catastróficas: si el sistema falla, se pierde toda la información y la disponibilidad de procesamiento.
Seguridad centralizada: un punto central de control facilita la implementación de políticas de seguridad.	Difícil recuperación: en caso de desastre, la recuperación y sincronización son complicadas.
Facilita la integridad: mantiene la precisión y consistencia de la información, sin necesidad de replicación o particionamiento de datos.	Cargas de trabajo no distribuidas: no se pueden difundir entre varios ordenadores.
Mejor rendimiento de procesamiento: centraliza y optimiza el procesamiento de datos.	Control centralizado: los departamentos de sistemas retienen todo el control.
Mantenimiento económico: requiere menos recursos humanos y optimiza el uso de recursos.	Mantenimiento central: requiere un mantenimiento central de datos.

En resumen, **aunque las *BD* centralizadas ofrecen ventajas como la eliminación de redundancia y mayor facilidad de mantenimiento, presentan desventajas significativas en términos de recuperación ante desastres y capacidad de procesamiento distribuido.**

8 *BD* distribuidas.

Una ***BD Distribuida (BDD)*** es un **conjunto de *BD* lógicamente relacionadas que se encuentran distribuidas en diferentes nodos interconectados por una red de comunicaciones.**

Un ***SGBD Distribuido (SGBDD)*** **administra la *BDD* y proporciona un acceso transparente a los usuarios**, haciendo que la distribución sea imperceptible. Desde un punto de vista lógico, funciona como si un solo *SGBD* en una sola máquina gestionara los datos.



Ventajas e inconvenientes de las *BD* distribuidas:

Ventajas	Inconvenientes
Acceso y procesamiento rápidos: varios nodos comparten la carga de trabajo, acelerando el procesamiento de datos.	Mayor riesgo de violaciones de seguridad: sin las precauciones adecuadas, la seguridad puede ser comprometida.
Acceso remoto: se puede acceder a información alojada en diferentes ubicaciones desde cualquier punto de la red.	Complejidad de coordinación: requiere mecanismos complejos para garantizar la sincronización adecuada entre nodos.
Costos iniciales más bajos: los servidores pequeños tienen costos iniciales menores comparados con los mainframes.	Mantenimiento costoso: aunque la inversión inicial es menor, el mantenimiento y control pueden ser caros.
Tolerancia a fallos: la replicación de datos permite que el sistema continúe funcionando si un nodo falla.	Control de concurrencia complejo: la replicación y distribución de datos complican el control de concurrencia y recuperación.

Adaptabilidad organizacional: se ajusta mejor a las estructuras organizativas descentralizadas y permite la incorporación flexible de nodos.

Independencia local: aunque interconectados, los nodos pueden funcionar de manera independiente.

Sobrecarga de coordinación: la comunicación entre nodos y el procesamiento adicional generan sobrecargas que no existen en sistemas centralizados.

Dificultad en asegurar la corrección: es complejo garantizar la corrección de los algoritmos y la recuperación en caso de fallos.

En resumen, las **BDD** ofrecen ventajas como mayor rapidez en el procesamiento y acceso remoto a los datos, así como una mejor adaptación a estructuras organizativas descentralizadas. Sin embargo, presentan desafíos significativos en términos de seguridad, complejidad de coordinación y mantenimiento.

8.1 Fragmentación.

En las **BDD**, la información se encuentra repartida en varios lugares. La forma de extraer los datos consultados puede realizarse mediante la fragmentación de distintas tablas pertenecientes a diferentes **BD** ubicadas en diversos servidores. La **fragmentación** se refiere al particionamiento de la información para distribuir cada parte a los distintos sitios de la red.

La fragmentación en BD consiste en dividir una BD grande en fragmentos más pequeños y distribuirlos en varios sistemas de almacenamiento o nodos de una red. Esto se hace para mejorar la eficiencia y el rendimiento de la BD, así como para aumentar la disponibilidad y la escalabilidad.

El grado de fragmentación es **crucial para la ejecución de consultas**. Si no hay fragmentación, las relaciones o tablas se tomarán como la unidad de fragmentación. La fragmentación **puede ser a nivel de tupla (fila o registro) o a nivel de atributo (columna o campo) de una tabla**. Un **grado de fragmentación** nulo o demasiado alto no es adecuado; el grado **debe estar equilibrado** y dependerá de las necesidades específicas de las aplicaciones que utilicen la **BD**. **El objetivo es encontrar un nivel de particionamiento adecuado entre tuplas, atributos y relaciones completas.**

Cuando se realiza una fragmentación, deben cumplirse tres **reglas fundamentales**:

- ✓ **Compleitud:** si una relación *R* se descompone en fragmentos *R1, R2, ..., Rn*, cada elemento de datos de *R* debe estar presente en al menos uno de los fragmentos *Ri*.
- ✓ **Reconstrucción:** la relación *R* debe poder reconstruirse a partir de sus fragmentos, preservando las restricciones definidas sobre los datos.
- ✓ **Disyunción:** si una relación *R* se descompone verticalmente, sus atributos primarios clave suelen repetirse en todos los fragmentos.

La fragmentación puede proporcionar varios **beneficios**, como **mejor utilización de los recursos de almacenamiento y procesamiento, mayor disponibilidad y resistencia a fallos, mejor escalabilidad y rendimiento mejorado** en consultas distribuidas. Sin embargo, también introduce **desafíos adicionales**, como la necesidad de **coordinar y sincronizar los fragmentos y garantizar la integridad de los datos entre ellos**.

Existen diferentes **métodos de fragmentación** en las **BD**, entre ellos:

- ✓ **Fragmentación horizontal:** se realiza sobre las tuplas o filas de la relación, dividiéndola en subrelaciones que contienen subconjuntos de las tuplas originales. Los fragmentos se definen mediante una operación de selección y se reconstruyen mediante la unión de los fragmentos componentes.

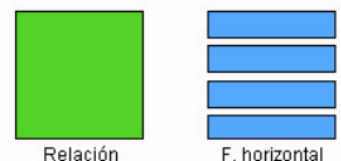
Existen dos variantes:

- **Primaria:** particiona las tuplas de una relación en subconjuntos con propiedades comunes.
- **Derivada:** divide una relación basándose en condiciones definidas sobre otra relación.

Los datos se dividen en filas y se distribuyen en múltiples nodos, cada uno conteniendo un subconjunto de las filas de la tabla original.

En la imagen se muestra una fragmentación horizontal por la columna *CD*, resultando en dos fragmentos primarios (*FH1* y *FH2*). Si la columna *CD* tuviera más valores distintos, se generarían tantos fragmentos como valores posibles.

Ejemplo: dada la tabla de la imagen realizar una fragmentación horizontal por provincia.



SNUM	NOMBRE	CD
S1	X	L
S2	Y	L
S3	Z	P
S4	A	P
S5	B	L

FH1 CD='L'		
SNUM	NOMBRE	CD
S1	X	L
S2	Y	L
S5	B	L

FH2 CD='P'		
SNUM	NOMBRE	CD
S3	Z	P
S4	A	P

En este ejemplo, la tabla se divide en tres fragmentos horizontales basados en la provincia. Los usuarios de la misma provincia se agrupan en el mismo nodo. Cada nodo es responsable de almacenar y gestionar los datos de los usuarios de una provincia específica.

Nodo 1:

FH1 Provincia='Madrid'

ID	Nombre	Apellido	Correo electrónico	Fecha de registro	Provincia
1	Juan	Pérez	juan@example.com	2022-01-01	Madrid
4	María	López	maria@example.com	2022-02-15	Madrid
7	Pedro	Ramírez	pedro@example.com	2022-03-10	Madrid

Nodo 3:

FH3 Provincia='Valencia'

ID	Nombre	Apellido	Correo electrónico	Fecha de registro	Provincia
3	Carlos	Torres	carlos@example.com	2022-01-10	Valencia
6	Laura	Sánchez	laura@example.com	2022-02-25	Valencia
9	Martín	Vargas	martin@example.com	2022-03-20	Valencia

ID	Nombre	Apellido	Correo electrónico	Fecha de registro	Provincia
1	Juan	Pérez	juan@example.com	2022-01-01	Madrid
2	Ana	García	ana@example.com	2022-01-05	Barcelona
3	Carlos	Torres	carlos@example.com	2022-01-10	Valencia
4	María	López	maria@example.com	2022-02-15	Madrid
5	Luis	Martínez	luis@example.com	2022-02-20	Barcelona
6	Laura	Sánchez	laura@example.com	2022-02-25	Valencia
7	Pedro	Ramírez	pedro@example.com	2022-03-10	Madrid
8	Andrea	Rodríguez	andrea@example.com	2022-03-15	Barcelona
9	Martín	Vargas	martin@example.com	2022-03-20	Valencia

Nodo 2:

FH2 Provincia='Barcelona'

ID	Nombre	Apellido	Correo electrónico	Fecha de registro	Provincia
2	Ana	García	ana@example.com	2022-01-05	Barcelona
5	Luis	Martínez	luis@example.com	2022-02-20	Barcelona
8	Andrea	Rodríguez	andrea@example.com	2022-03-15	Barcelona

Ejemplo: fragmentación horizontal derivada por provincia en dos tablas relacionadas (usuarios y pedidos).

Tabla "Usuarios":

ID	Nombre	Apellido	Correo electrónico	Provincia
1	Juan	Pérez	juan@example.com	Madrid
2	Ana	García	ana@example.com	Barcelona
3	Carlos	Torres	carlos@example.com	Valencia
4	María	López	maria@example.com	Madrid
5	Luis	Martínez	luis@example.com	Barcelona
6	Laura	Sánchez	laura@example.com	Valencia
7	Pedro	Ramírez	pedro@example.com	Madrid
8	Andrea	Rodríguez	andrea@example.com	Barcelona
9	Martín	Vargas	martin@example.com	Valencia

Tabla "Pedidos":

ID	Usuario_ID	Producto	Cantidad
1	1	A	5
2	2	B	3
3	3	C	2
4	4	A	4
5	5	B	1
6	6	C	2
7	7	A	6
8	8	B	2
9	9	C	3

Para realizar la fragmentación horizontal derivada, se va a dividir los datos de las tablas "usuarios" y "pedidos" en tres nodos basados en la provincia. A continuación, se muestra cómo quedarían los fragmentos:

Nodo 1:

Tabla "Usuarios": FH1 Provincia='Madrid'

ID	Nombre	Apellido	Correo electrónico	Provincia
1	Juan	Pérez	juan@example.com	Madrid
4	María	López	maria@example.com	Madrid
7	Pedro	Ramírez	pedro@example.com	Madrid

Nodo 2:

Tabla "Usuarios": FH2 Provincia='Barcelona'

ID	Nombre	Apellido	Correo electrónico	Provincia
2	Ana	García	ana@example.com	Barcelona
5	Luis	Martínez	luis@example.com	Barcelona
8	Andrea	Rodríguez	andrea@example.com	Barcelona

Nodo 3:

Tabla "Usuarios": FH3 Provincia='Valencia'

ID	Nombre	Apellido	Correo electrónico	Provincia
3	Carlos	Torres	carlos@example.com	Valencia
6	Laura	Sánchez	laura@example.com	Valencia
9	Martín	Vargas	martin@example.com	Valencia

Tabla "Pedidos": FD1 Provincia='Madrid'

ID	Usuario_ID	Producto	Cantidad
1	1	A	5
4	4	A	4
7	7	A	6

Tabla "Pedidos": FD2 Provincia='Barcelona'

ID	Usuario_ID	Producto	Cantidad
2	2	B	3
5	5	B	1
8	8	B	2

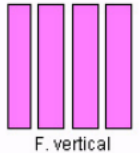
Tabla "Pedidos": FD3 Provincia='Valencia'

ID	Usuario_ID	Producto	Cantidad
3	3	C	2
6	6	C	2
9	9	C	3

En esta fragmentación derivada, cada nodo contiene una combinación de usuarios y pedidos basados en la provincia. Esto permite una distribución equilibrada de los datos relacionados entre los nodos, considerando

tanto la información de usuarios como los pedidos asociados a ellos en función de la provincia a la que pertenecen.

- ✓ **Fragmentación vertical:** la fragmentación vertical se basa en la división de una relación en fragmentos según sus atributos. En este proceso, una relación R se divide en varios fragmentos $R_1, R_2, \dots, R_r$, donde cada fragmento contiene un subconjunto de los atributos de R , junto con la clave primaria de R .



El principal objetivo de la fragmentación vertical es crear relaciones más pequeñas para optimizar la ejecución de las aplicaciones de usuario. Una fragmentación óptima es aquella que produce un esquema que minimiza el tiempo de ejecución de las consultas. La fragmentación vertical es más compleja que la horizontal debido a las múltiples opciones posibles para dividir los atributos.

En la imagen se muestra una fragmentación vertical de una tabla empleados, dividiéndola por los atributos *NOMBRE*, *JEFE* y *DEPT* en un fragmento, y *SALARIO* e *IMPTO* en otro fragmento. Los fragmentos resultantes son FV_1 y FV_2 , y es importante que el atributo clave (en este caso, *EMP#*) aparezca en todos los fragmentos para mantener la integridad referencial.

EMP#	NOMBRE	SALARIO	IMPTO	JEFE	DEPT
e1	X	1000	100	J1	D1
e2	Y	1500	300	J1	D1
e3	Z	500	20	J2	D2
e4	A	4000	1000	J3	D3
e5	B	2000	350	J2	D2

EMP#	NOMBRE	JEFE	DEPT
e1	X	J1	D1
e2	Y	J1	D1
e3	Z	J2	D2
e4	A	J3	D3
e5	B	J2	D2

EMP#	SALARIO	IMPTO
e1	1000	100
e2	1500	300
e3	500	20
e4	4000	1000
e5	2000	350

$FV_1 = E[EMP\#, NOMBRE, JEFE, DEPT]$ $FV_2 = E[EMP\#, SALARIO, IMPTO]$

En la fragmentación vertical, los datos se dividen en columnas y se distribuyen entre múltiples nodos. Cada nodo contiene un subconjunto de las columnas de la tabla original, permitiendo un acceso más eficiente y focalizado a los datos según las necesidades específicas de las aplicaciones de usuario.

Ejemplo: fragmentación vertical de la tabla usuarios.

En este ejemplo, el Fragmento vertical 1 contiene las columnas "ID", "Nombre" y "Apellido", mientras que el Fragmento vertical 2 contiene las columnas "ID" y "Dirección". Cada fragmento muestra una parte de la información original de la tabla.

ID #	Nombre	Apellido	Dirección
1	Juan	Pérez	Calle 123
2	María	López	Avenida 456
3	Carlos	Rodríguez	Calle 789
4	Ana	Gómez	Avenida 012

Fragmentación vertical 1: Fragmento 1

FV1 = Usuarios [Nombre, Apellido]

ID #	Nombre	Apellido
1	Juan	Pérez
2	María	López
3	Carlos	Rodríguez
4	Ana	Gómez

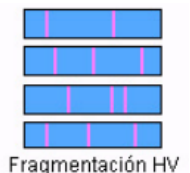
Fragmentación vertical 2: Fragmento 2

FV2 = Usuarios [Dirección]

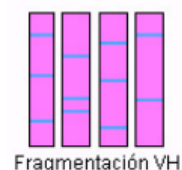
ID #	Dirección
1	Calle 123
2	Avenida 456
3	Calle 789
4	Avenida 012

- ✓ **Fragmentación híbrida o mixta:** combina la fragmentación horizontal y vertical para crear fragmentos más específicos y eficientes. Existen tres tipos principales de fragmentación mixta:

- **Fragmentación mixta HV:** consiste en aplicar primero una fragmentación horizontal y luego una vertical a cada uno de los fragmentos horizontales. Esto permite segmentar los datos inicialmente por filas y luego dividir cada segmento por columnas.



- **Fragmentación mixta VH:** se realiza aplicando primero una fragmentación vertical y luego una horizontal. En este caso, los datos se dividen primero por columnas y luego cada conjunto de columnas se segmenta por filas.



- **Celdas:** se aplican simultáneamente la fragmentación horizontal y vertical, formando una red de celdas. Este tipo de fragmentación produce fragmentos donde todos los datos obtenidos son iguales, creando una estructura uniforme de celdas.



Ejemplo: fragmentación mixta HV de la tabla empleados.

Para realizar una fragmentación mixta *HV* de la tabla de empleados, hay que aplicar una fragmentación horizontal primero y seguidamente una fragmentación vertical a los fragmentos horizontales obtenidos.

Fragmentación horizontal → se dividirá la tabla en tres fragmentos horizontales basados en el atributo Departamento.

Fragmento 1 - Marketing:

ID #	Nombre	Departamento	Puesto	Salario
1	John Smith	Marketing	Gerente	\$5000
5	Alex Lee	Marketing	Asistente	\$2500

Fragmento 3 - Finanzas:

ID #	Nombre	Departamento	Puesto	Salario
3	Mike Brown	Finanzas	Analista	\$3500

Fragmento 2 - Ventas:

ID #	Nombre	Departamento	Puesto	Salario
2	Jane Doe	Ventas	Ejecutivo	\$4000
4	Lisa Green	Ventas	Representante	\$3000

Tabla: Empleados

ID #	Nombre	Departamento	Puesto	Salario
1	John Smith	Marketing	Gerente	\$5000
2	Jane Doe	Ventas	Ejecutivo	\$4000
3	Mike Brown	Finanzas	Analista	\$3500
4	Lisa Green	Ventas	Representante	\$3000
5	Alex Lee	Marketing	Asistente	\$2500

Si se desea realizar una fragmentación vertical a los tres fragmentos horizontales obtenidos anteriormente, teniendo en cuenta todas las filas, se puede dividir cada fragmento en dos fragmentos verticales basados en los atributos: Nombre/Departamento/Puesto y Nombre/Salario.

Fragmento 1 - Marketing:

ID #	Nombre	Departamento	Puesto
1	John Smith	Marketing	Gerente
5	Alex Lee	Marketing	Asistente

ID #	Nombre	Salario
1	John Smith	\$5000
5	Alex Lee	\$2500

Fragmento 2 - Ventas:

ID #	Nombre	Departamento	Puesto
2	Jane Doe	Ventas	Ejecutivo
4	Lisa Green	Ventas	Representante

ID #	Nombre	Salario
2	Jane Doe	\$4000
4	Lisa Green	\$3000

Fragmento 3 - Finanzas:

ID #	Nombre	Departamento	Puesto
3	Mike Brown	Finanzas	Analista

ID #	Nombre	Salario
3	Mike Brown	\$3500

Es importante tener en cuenta que la fragmentación mixta puede variar según las necesidades y consultas específicas del sistema. El ejemplo proporcionado muestra una posible forma de realizar la fragmentación vertical, pero la elección de los atributos y cómo se dividen depende de los requisitos y objetivos particulares del sistema.

- ✓ **Fragmentación funcional:** los **datos se dividen en fragmentos según algún criterio o función específica**. Por ejemplo, los datos pueden fragmentarse según la ubicación geográfica, la categoría de los usuarios, o cualquier otro criterio relevante para la aplicación.

La fragmentación funcional es especialmente útil en casos donde los datos tienen una distribución natural o lógica que puede aprovecharse para mejorar el rendimiento y la gestión de la *BD*. Además, facilita la escalabilidad y la optimización de las consultas al permitir que cada fragmento contenga datos relacionados y coherentes entre sí.

Ejemplo: fragmentación funcional de la tabla productos.

Se quiere fragmentar esta tabla en un entorno distribuido considerando las funciones más frecuentes en el sistema:

- Fragmento 1 - Consulta de precios: este fragmento contendrá solo los atributos *ID*, *Nombre*, *Categoría* y *Precio* de los productos.
- Fragmento 2 - Consulta de stock: este fragmento contendrá solo los atributos *ID*, *Nombre*, *Categoría* y *Stock* de los productos.

Al fragmentar la tabla de esta manera, se mejora el rendimiento al realizar consultas específicas sin tener que acceder a todos los atributos de la tabla original. Por ejemplo, si se necesita consultar los precios, solo se necesita acceder al Fragmento 1, que contiene los datos necesarios para esta función. De manera similar, si se requiere consultar el stock, se puede acceder directamente al Fragmento 2.

Es crucial diseñar la fragmentación funcional cuidadosamente para reflejar las consultas más frecuentes y garantizar que los fragmentos resultantes contengan la información necesaria para satisfacer esas consultas. Además, puede ser necesario implementar mecanismos de coordinación y sincronización entre

Tabla: Productos

ID #	Nombre	Categoría	Precio	Stock
1	Producto A	Categoría1	\$10	100
2	Producto B	Categoría2	\$20	150
3	Producto C	Categoría1	\$15	80
4	Producto D	Categoría3	\$25	120
5	Producto E	Categoría2	\$18	90

los fragmentos para garantizar la consistencia de los datos en el sistema distribuido.

La fragmentación funcional de la tabla "Productos" quedaría de la siguiente forma:

Fragmento 1 - Consulta de precios:

ID #	Nombre	Categoría	Precio
1	Producto A	Categoría1	\$10
2	Producto B	Categoría2	\$20
3	Producto C	Categoría1	\$15
4	Producto D	Categoría3	\$25
5	Producto E	Categoría2	\$18

Fragmento 2 - Consulta de stock:

ID #	Nombre	Categoría	Stock
1	Producto A	Categoría1	100
2	Producto B	Categoría2	150
3	Producto C	Categoría1	80
4	Producto D	Categoría3	120
5	Producto E	Categoría2	90

TAREA

12. ¿Qué permiten las **BD distribuidas**? Enumerar sus ventajas e inconvenientes.

13. Considerar la siguiente relación, ¿qué fragmentos se obtendrían al realizar una fragmentación horizontal por **ESCUELA**?

Jno	NOMBRE	NOTA	ESCUELA
J1	LUIS YANEZ	8	CIME
J2	ERIKA QUIROZ	8	CIME
J3	DANIEL MURILLO	9	EISIC
J4	MARIA JOSE MENDEZ	10	EISIC
J5	ADONIS PABON	9	EISIC

14. Considerar las siguientes relaciones, ¿qué fragmentos principales y derivados se obtendrían al realizar una fragmentación horizontal por **ESCUELA**?

ID	MATERIA	ESCUELA	CRÉDITOS	NIVEL
1	Análisis Matemático	EISIC	4	1
2	Sistemas Operativos	EISIC	6	3
3	Programación II	CIME	6	2
4	Tecnología Eléctrica	EISIC	4	2
5	Técnicas de Aprendizaje	CIME	4	1
6	Dibujo Mecánico	CIME	6	3

Jno	NOMBRE	MATERIA	NOTA
J1	LUIS YANEZ	Sistemas Operativos	8
J2	ERIKA QUIROZ	Dibujo Mecánico	8
J3	DANIEL MURILLO	Técnicas de Aprendizaje	9
J4	MARIA JOSE MENDEZ	Análisis Matemático	10
J5	ADONIS PABON	Programación II	9

15. Considerar la relación del ejercicio 13. Realizar una fragmentación vertical por **NOTA** y por **NOMBRE/ESCUELA**.

16. Realizar una fragmentación mixta **HV**. La fragmentación horizontal se debe realizar por **ESCUELA** y la vertical por **ESCUELA/NOTA_INGRESO/NOMBRE** y **NOMBRE/BECA**.

DNI	Escuela	Nombre	Nota ingreso	Beca
87633483	EUI	Concha Queta	5.6	No
99855743	EUI	Josechu Letón	7.2	Si
33887293	EUIT	Oscar Romato	6.1	Si
05399075	EUI	Bill Gates	5.0	No
44343234	EUIT	Pepe Pótamo	8.0	No
44543324	EUI	Maite Clado	7.5	Si
66553234	EUIT	Ernesto Mate	6.6	No

9 Legislación sobre protección de datos.

Todas las **personas físicas y jurídicas** (ya sean públicas o privadas) que traten o manejen datos de carácter personal tienen que cumplir con una serie de principios y obligaciones, legal y reglamentariamente establecidos, que garanticen, en último término, el derecho de las personas a controlar y disponer de sus datos personales, esto es, su derecho a la protección de datos personales.

Un **dato de carácter personal** es toda aquella **información que se refiere a una persona física identificada o identificable**, desde su nombre y apellidos, domicilio, **DNI**, hasta cualquier otra que revele información sobre sus hábitos, preferencias o forma de vida.

Los **datos personales que hayan sido anonimizados**, de forma que la persona no sea identificable o deje de serlo, **dejarán de considerarse datos personales**. Para que los datos se consideren verdaderamente anónimos, la anonimización debe ser irreversible.

Son **datos especialmente protegidos** los que revelen “el **origen étnico o racial**, las **opiniones políticas**, las **convicciones religiosas o filosóficas**, o la **afiliación sindical**, y el **tratamiento de datos genéticos**, **datos biométricos** dirigidos a identificar de manera unívoca a una persona física, **datos relativos a la salud** o **datos relativos a la vida sexual** o la **orientación sexual** de una persona física”. Como regla general, **para el tratamiento de estos datos se requerirá el consentimiento expreso y por escrito de su titular**.



La legislación sobre protección de datos en España se centra principalmente en el cumplimiento del **Reglamento General de Protección de Datos (RGPD)** de la **Unión Europea** y la **Ley Orgánica de Protección de Datos y Garantía de los Derechos Digitales (LOPDGDD)**.

La Protección de Datos de Carácter Personal es un derecho fundamental que tienen todas las personas físicas para garantizar su vida privada y supone el control y poder de disposición de la información relativa a su esfera tanto pública como privada. Se garantiza por el artículo 18.4 de la Constitución Española.

9.1 Reglamento General de Protección de Datos (RGPD).

El **Reglamento General de Protección de Datos (Reglamento (UE) 2016/679)** es una legislación de la **Unión Europea** que **entró en vigor el 25 de mayo de 2018**. Se aplica directamente a todos los estados miembros de la **UE**, incluida España, y **establece un marco común para la protección de los datos personales**.

La **nueva normativa comunitaria centra su atención** no en los ficheros de datos sino **en los tratamientos efectuados con los datos personales** incluidos en dichos ficheros. El **RGPD suprime la obligación de los responsables de los tratamientos de datos de inscribir o notificar ficheros**. Ahora, sobre los mismos recae la **obligación de que los mismos lleven un registro de actividades o de los tratamientos efectuados**.

Sus **principales puntos** son:

1. Ámbito de aplicación.

- Se **aplica a todas las empresas y organizaciones que procesan datos personales de residentes en la UE**, independientemente de su ubicación.
- **Incluye tanto a controladores** (quienes determinan los fines y medios del procesamiento de datos) **como a procesadores** (quienes procesan datos en nombre del controlador).

2. Principios relativos al tratamiento de datos personales.

- Licitud, lealtad y transparencia:** los datos deben ser procesados de manera legal, justa y transparente.
- Limitación de la finalidad:** los datos solo deben ser recogidos para fines específicos, explícitos y legítimos.
- Minimización de datos:** solo se deben recoger los datos necesarios para el propósito específico.
- Exactitud:** los datos deben ser exactos y estar actualizados.
- Limitación del plazo de conservación:** los datos no deben conservarse más tiempo del necesario.
- Integridad y confidencialidad:** los datos deben ser procesados de manera segura.
- Responsabilidad proactiva:** el responsable del tratamiento debe cumplir con todos los principios del tratamiento de datos personales y, lo que es más importante, debe ser "capaz de demostrarlo".

3. Derechos de los interesados.

- De **acceso:** los individuos pueden solicitar acceso a sus datos personales.
- De **rectificación:** los individuos pueden solicitar la corrección de datos inexactos.
- De **supresión** ("derecho al olvido"): los individuos pueden solicitar la eliminación de sus datos personales.
- De **limitación del tratamiento:** los individuos pueden solicitar restricciones en el procesamiento de sus datos.
- De **portabilidad de los datos:** los individuos pueden solicitar que sus datos sean transferidos a otro controlador.
- De **oposición:** los individuos pueden oponerse al procesamiento de sus datos.
- De **no ser objeto de decisiones automatizadas:** los individuos tienen derecho a no ser objeto de decisiones basadas únicamente en procesamiento automatizado.
- De **indemnización:** posibilidad de indemnización en el caso de que el interesado vea lesionado su derecho a la protección de datos o por un tratamiento contrario a la normativa vigente.

4. Obligaciones de las organizaciones.

- **Consentimiento:** debe obtenerse un consentimiento explícito y verificable para el procesamiento de datos.
- **Evaluaciones de impacto de protección de datos:** deben realizarse evaluaciones de impacto cuando el procesamiento puede resultar en un alto riesgo para los derechos y libertades de los individuos.
- **Delegado de Protección de Datos (DPO):** se debe designar un **DPO** en ciertos casos, como cuando el procesamiento es llevado a cabo por una autoridad pública o implica una monitorización sistemática a gran

escala.

- **Notificación de violaciones de datos:** las violaciones de datos deben notificarse a la autoridad de supervisión dentro de las 72 horas y, en algunos casos, a los individuos afectados.
- **Registro de las actividades de tratamiento.**

Acceso al texto completo del RGPD en español: <https://www.boe.es/doue/2016/119/L00001-00088.pdf>

9.2 Ley Orgánica de Protección de Datos y Garantía de los Derechos Digitales (LOPDGDD).

La Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y Garantía de los Derechos Digitales (LOPDGDD), adapta el RGPD a la legislación española y añade disposiciones específicas para garantizar los derechos digitales. Los puntos clave de la LOPDGDD son:

1. Adaptación al RGPD.

La LOPDGDD complementa y desarrolla las disposiciones del RGPD, proporcionando detalles específicos sobre su aplicación en España.

2. Derechos digitales.

La ley incluye un título dedicado a la protección de los **derechos** digitales. Algunos de ellos son:

- **A la protección de datos personales:** garantiza a las personas el control sobre sus datos personales y la capacidad de decidir cómo se recopilan, utilizan y comparten en el entorno digital.
- **Al olvido digital:** permite a las personas solicitar la eliminación de información personal en línea que ya no sea relevante, precisa o necesaria para los fines para los cuales se recopiló.
- **A la privacidad en el entorno digital:** protege la privacidad de las personas en el entorno digital, incluida la protección de la confidencialidad de las comunicaciones electrónicas y la prevención del acceso no autorizado a datos personales.
- **A la neutralidad de la red:** garantiza que todos los datos en línea sean tratados de manera igualitaria, sin discriminación ni restricciones injustas por parte de proveedores de servicios de Internet u otros actores.
- **A la seguridad digital:** protege a las personas contra amenazas a la seguridad en línea, como el robo de identidad, el fraude electrónico y otras formas de ciberdelincuencia.
- **A la educación digital:** garantiza el acceso universal a la educación y la alfabetización digital, con el objetivo de promover el uso seguro, ético y responsable de la tecnología.
- **A la desconexión digital:** protege el tiempo de descanso y el equilibrio entre la vida laboral y personal, garantizando que los trabajadores no estén constantemente disponibles fuera del horario laboral y tengan derecho a desconectar del trabajo.

En conjunto, estos derechos buscan proteger los derechos fundamentales de los ciudadanos en el entorno digital y promover un uso seguro, ético y responsable de la tecnología.

3. Delegado de Protección de Datos (DPO).

La LOPDGDD establece las condiciones específicas bajo las cuales debe designarse un DPO en España:

- Las **autoridades u organismos públicos, excepto los tribunales** que actúen en ejercicio de su función judicial.
- Las **empresas u organizaciones cuyas actividades principales consistan en el tratamiento a gran escala de categorías especiales de datos** (datos sensibles) **o datos relativos a condenas e infracciones penales**.
- Las **empresas u organizaciones que realicen el monitoreo regular y sistemático a gran escala de personas**.

Además, en España, el resto de **organizaciones pueden optar por designar un DPO de manera voluntaria**, incluso si no están obligadas por la ley. La designación de un DPO es una medida recomendada para garantizar el cumplimiento de las normativas de protección de datos y promover una cultura de privacidad dentro de la organización.

4. Autoridad de supervisión.

La Agencia Española de Protección de Datos (AEPD) es la autoridad encargada de supervisar y garantizar el cumplimiento de la normativa de protección de datos en España.

5. Medidas de seguridad.

La ley establece requisitos específicos sobre las medidas de seguridad que deben implementarse para proteger los datos personales. Algunas de las **medidas** de seguridad que deben ser implementadas según la *LOPDGDD* son:

- Organizativas:
 - Desarrollo y aplicación de políticas y procedimientos internos para la protección de datos personales.
 - Designación de un *DPO* en aquellos casos en los que sea obligatorio según la ley o de forma voluntaria para garantizar el cumplimiento de la normativa.
- Técnicas:
 - Encriptación de los datos personales para garantizar su confidencialidad durante la transmisión y almacenamiento.
 - Implementación de controles de acceso y autenticación para garantizar que solo las personas autorizadas tengan acceso a los datos.
 - Utilización de firewalls, sistemas de detección de intrusos y otras herramientas de seguridad informática para proteger los sistemas y redes de la organización contra amenazas externas.
 - Realización de copias de seguridad periódicas de los datos personales para garantizar su disponibilidad en caso de incidentes o pérdida de información.
- Físicas:
 - Implementación de medidas de seguridad física, como sistemas de control de acceso, cámaras de vigilancia y cerraduras, para proteger los equipos y dispositivos que contienen datos personales.
 - Restricción del acceso físico a las instalaciones donde se almacenan o procesan los datos personales.
- Legales:
 - Establecimiento de contratos de tratamiento de datos con terceros que puedan tener acceso a los datos personales en nombre de la organización, para garantizar que cumplan con las mismas medidas de seguridad requeridas por la *LOPDGDD*.
 - Notificación de violaciones de seguridad de datos a la *AEPD* y a los interesados afectados en caso de que se produzca una brecha de seguridad que pueda afectar a la protección de datos personales.

Sanciones y cumplimiento.

El incumplimiento del *RGPD* y la *LOPDGDD* puede resultar en sanciones significativas. Las multas pueden llegar hasta el 4% de la facturación anual global de la empresa o 20 millones de euros, lo que sea mayor. La *AEPD* es responsable de imponer estas sanciones y de proporcionar orientación sobre el cumplimiento de la normativa.

Las infracciones **se dividen en:**

- ✓ Muy graves: prescriben a los tres años.
 - Uso de los datos para una finalidad diferente de la anunciada.
 - Omisión del deber de informar al afectado.
 - Exigencia de un pago para poder acceder a los datos propios almacenados.
 - Transferencia internacional de información sin garantías.
- ✓ Graves: prescriben a los dos años.
 - Datos de un menor recabados sin consentimiento.
 - Falta de adopción de medidas técnicas y organizativas necesarias para la efectiva protección de datos.
 - Incumplimiento del deber de nombrar responsable o encargado de tratamiento de datos.
- ✓ Leves: prescriben en un año.
 - No transparencia de la información.
 - Incumplimiento de no informar al afectado cuando lo haya solicitado.
 - Incumplimiento por parte del encargado.

Recursos y herramientas.

- *AEPD*: proporciona recursos y herramientas para ayudar a las organizaciones a cumplir con la normativa de protección de datos. Su sitio web (<https://www.aepd.es/>) incluye guías, formularios y una herramienta de evaluación de riesgos.
- Documentación: el texto completo del *RGPD* está disponible en línea, y existen numerosas guías y comentarios que explican sus disposiciones en detalle.

- Formación y consultoría: existen numerosas empresas y profesionales que ofrecen formación y servicios de consultoría para ayudar a las organizaciones a implementar y mantener el cumplimiento de la normativa de protección de datos.

Acceso al texto completo de la LOPDGDD: <https://www.boe.es/buscar/pdf/2018/BOE-A-2018-16673-consolidado.pdf>

10 Big Data.

10.1 Introducción.

Big Data se refiere a la gestión y análisis de grandes volúmenes de datos que son demasiado complejos para ser manejados con herramientas tradicionales de procesamiento de datos. Estos datos pueden ser estructurados, semiestructurados o no estructurados, y provienen de diversas fuentes como redes sociales, dispositivos IoT, transacciones comerciales, etc.



Características (las 5 V's):

- ✓ **Volumen:** cantidad de datos que se generan y almacenan. Enorme crecimiento de datos en términos de terabytes, petabytes, etc.
- ✓ **Velocidad:** rapidez con la que se generan y procesan los datos. Requiere procesamiento en tiempo real.
- ✓ **Variedad:** diversidad de tipos de datos (texto, imágenes, videos, datos de sensores, etc.).
- ✓ **Veracidad:** calidad y precisión de los datos.
- ✓ **Valor:** potencial de los datos para generar valor y obtener *insights* (descubrimientos o interpretaciones valiosas derivadas de la observación y el análisis de datos) útiles.

Componentes:

- ✓ **Fuentes de datos:** sensores, redes sociales, transacciones, etc.
- ✓ **Almacenamiento de datos:** BD NoSQL, sistemas de archivos distribuidos como *Hadoop HDFS*.
- ✓ **Procesamiento de datos:** herramientas como *Hadoop, Spark, Storm*.
- ✓ **Análisis de datos:** herramientas y técnicas para extraer información útil de los datos (análisis descriptivo, predictivo y prescriptivo).

10.2 Clasificación de los datos.

En el contexto de *Big Data*, los datos pueden clasificarse en tres categorías principales:

- ✓ **Datos estructurados:** son aquellos que están **organizados en un formato predefinido y fijo**, generalmente en tablas con filas y columnas. Estos datos son fáciles de almacenar y analizar utilizando SGBD relacionales.

Características:

- Formato predefinido: tienen una estructura fija con esquemas definidos (columnas, tipos de datos).
- Acceso rápido: permiten un acceso rápido y eficiente mediante consultas SQL.
- Facilidad de análisis: son fáciles de analizar debido a su formato tabular y bien definido.

- ✓ **Datos semiestructurados:** tienen una **estructura que no es rígida** como la de los datos estructurados. Aunque contienen etiquetas o marcadores para separar elementos de datos, no siguen un formato fijo como las tablas. Ejemplos: documentos XML, JSON, ...

Características:

- Flexibilidad: pueden tener una estructura variable y no requieren un esquema fijo.
- Auto-descriptivos: contienen metadatos que describen la estructura y los datos.
- Interoperabilidad: son fácilmente intercambiables entre diferentes sistemas y lenguajes de programación.

- ✓ **Datos no estructurados:** **no tienen una estructura predefinida ni organizada**. Estos datos no se ajustan fácilmente a los modelos tabulares y son difíciles de analizar sin un procesamiento previo. Ejemplos: documentos de texto (*Word, PDF*), archivos multimedia (imágenes, videos, audios), publicaciones en redes

sociales, etc.

Características:

- Formato libre: no tienen un formato ni esquema fijo.
- Volumen grande: suelen generar grandes volúmenes de datos, como en el caso de videos o imágenes de alta resolución.
- Procesamiento complejo: requieren técnicas avanzadas de procesamiento y análisis, como minería de datos, análisis de texto y aprendizaje automático.

10.3 Análisis de datos.

El análisis de datos en el contexto de *Big Data* implica la exploración, limpieza, transformación y modelado de grandes conjuntos de datos con el objetivo de descubrir información útil, extraer conclusiones y apoyar la toma de decisiones.

Tipos:

- ✓ **Análisis descriptivo:** describe lo que ha sucedido. Herramientas como *dashboards* (tableros de control), informes y estadísticas.
- ✓ **Análisis diagnóstico:** investiga por qué algo ha sucedido. Incluye técnicas de minería de datos y análisis de correlación.
- ✓ **Análisis predictivo:** predice lo que es probable que suceda en el futuro. Utiliza técnicas de machine learning y modelos estadísticos.
- ✓ **Análisis prescriptivo:** recomienda acciones a tomar. Se basa en análisis predictivo y modelos de optimización.

Herramientas y tecnologías:

- ✓ **Apache Hadoop:** framework para almacenamiento y procesamiento distribuido de grandes conjuntos de datos.
- ✓ **Apache Spark:** motor de procesamiento rápido y general para grandes conjuntos de datos.
- ✓ **BD NoSQL:** BD no relacionales como *MongoDB*, *Cassandra*, que permiten manejar datos no estructurados.
- ✓ **Librerías de Machine Learning:** como *Scikit-learn*, *TensorFlow*, *PyTorch* para crear modelos predictivos.

Proceso:

- ✓ **Recolección de datos:** obtener datos de diferentes fuentes.
- ✓ **Almacenamiento:** utilizar sistemas de almacenamiento para manejar grandes volúmenes de datos.
- ✓ **Limpieza y preparación:** eliminar inconsistencias y preparar los datos para el análisis.
- ✓ **Exploración y visualización:** utilizar herramientas para explorar y visualizar los datos.
- ✓ **Modelado:** aplicar técnicas de análisis estadístico y *machine learning*.
- ✓ **Interpretación y comunicación:** interpretar los resultados y comunicar los insights obtenidos.

10.4 Inteligencia de negocios.

La inteligencia de negocios es una parte del análisis de datos que se enfoca específicamente en proporcionar información relevante para la toma de decisiones empresariales, utilizando técnicas de análisis de datos para alcanzar este objetivo.

Inteligencia de negocios (**Business Intelligence, BI**) se refiere a las estrategias y tecnologías utilizadas por las empresas para el análisis de datos de negocio. El objetivo de la **BI** es transformar los datos en información valiosa y procesable para mejorar la toma de decisiones empresariales.



Componentes:

- ✓ **Data Warehousing:** almacenamiento centralizado de datos integrados de diferentes fuentes. Ejemplos: *Amazon Redshift*, *Google BigQuery*.
- ✓ **ETL (Extract, Transform, Load):** proceso de extracción de datos de diversas fuentes, transformación a un formato adecuado y carga en un data warehouse.

- ✓ **Herramientas de BI:** herramientas que permiten la visualización y análisis de datos. Ejemplos: *Tableau*, *Power BI*, *QlikView*.
- ✓ **Informes y dashboards:** interfaces que permiten a los usuarios ver y analizar datos fácilmente.

Proceso:

1. **Recolección de datos:** recopilar datos de diversas fuentes de la empresa.
2. **Integración de datos:** combinar datos de diferentes fuentes en un almacén de datos.
3. **Análisis de datos:** utilizar herramientas de *BI* para analizar los datos y obtener *insights*.
4. **Visualización de datos:** crear informes y *dashboards* interactivos para visualizar los datos de manera clara.
5. **Toma de decisiones:** utilizar los *insights* obtenidos para apoyar la toma de decisiones estratégicas y operativas.

Beneficios:

- ✓ **Mejora en la toma de decisiones:** proporciona datos precisos y actualizados para apoyar decisiones.
- ✓ **Optimización de procesos:** identifica áreas de mejora en los procesos empresariales.
- ✓ **Mayor eficiencia operativa:** permite una mejor gestión de recursos y operaciones.
- ✓ **Ventaja competitiva:** ayuda a identificar tendencias y oportunidades en el mercado.

Desafíos:

- ✓ **Integración de datos:** unificar datos de diferentes sistemas y fuentes.
- ✓ **Calidad de datos:** asegurar que los datos sean precisos y consistentes.
- ✓ **Seguridad:** proteger los datos contra accesos no autorizados y brechas de seguridad.
- ✓ **Escalabilidad:** manejar el crecimiento de los datos y la complejidad del análisis a medida que la empresa crece.

Big Data, análisis de datos e inteligencia de negocios son áreas interrelacionadas que permiten a las empresas aprovechar grandes volúmenes de datos para obtener insights valiosos, optimizar procesos y tomar decisiones estratégicas basadas en datos.



TAREA

17. ¿Qué es el *RGPD* y cuál es su objetivo principal? ¿qué es la *LOPDGDD* y cómo se relaciona con el *RGPD*?
18. ¿Qué implica el principio de minimización de datos y cómo debe aplicarse en una organización?
19. ¿Qué es el "derecho al olvido"? ¿en qué circunstancias puede ser invocado?
20. ¿Qué medidas deben tomar las organizaciones para garantizar el cumplimiento del *RGPD*? ¿qué es el *DPO* y en qué casos es obligatoria su designación?
21. Explicar el derecho a la desconexión digital y su relevancia en el entorno laboral.
22. ¿Qué es el *Big Data* y cuáles son las características principales que lo definen? Relacionar las características del *Big Data* con los principios y derechos establecidos por el *RGPD*.
23. ¿Qué es la inteligencia de negocios y cómo difiere del análisis de datos? ¿cuáles son los principales beneficios de implementar un sistema de *BI* en una organización?